MACHINE LEARNING & GENAI INTERVIEW PREPARATION

Part 7: LLM Inference & Optimization

Decoding Mechanics, KV Cache Engineering, Model Quantization, Speculative Verification, Serving Systems, and Incident Debugging Study Guide

Target Persona: Senior AI Engineer / LLM Systems Architect (~3 YOE)

Core Modules: Inference Fundamentals, Decoding & Sampling, KV Cache Optimization, Model Quantization (GPTQ/AWQ/SmoothQuant), Iteration-Level Continuous Batching, Speculative Decoding, Serving Engine Stacks, Production Gateway Resilience, Performance Telemetry, and Dynamic Troubleshooting.

Includes: Mathematical proof of speculative rejection sampling lossless properties, arithmetic intensity Roofline ridge-point calculations, asymmetric quantization zero-points derivations, GQA compression VRAM estimations, Radix tree prefix caches matching steps, and p99 latency SLA incident checklists.

Table of Contents

01. Inference Fundamentals: Prefill, Decode, and Roofline Mechanics

Autoregressive Large Language Model (LLM) serving presents a unique hardware and systems engineering challenge. Unlike traditional deep learning models where training and inference share similar compute-bound profiles, LLM inference changes bottlenecks dynamically between its execution phases. Understanding these bottlenecks is critical for optimizing throughput, satisfying latency Service Level Agreements (SLAs), and designing high-scale serving infrastructure.

---

1. Training vs. Inference Bottlenecks

An LLM's life cycle is split into two compute paradigms: training (which is compute-bound) and autoregressive inference (which is memory-bandwidth-bound during token generation).

Training Phase (Compute-Bound)

During pre-training or fine-tuning, the model processes large batches of sequences simultaneously. The training step utilizes the *causal mask* to calculate self-attention across all tokens in a sequence concurrently:
- We compute projections and attention scores for all sequence positions at once.
- Weights are loaded into SRAM once and reused to compute activations for thousands of tokens in parallel.
- The GPU Tensor Cores are kept highly saturated, making training efficiency scale with the hardware's peak FLOP/s capacity.

Inference Phase (Autoregressive Decode is Memory-Bandwidth-Bound)

Autoregressive inference generates text token-by-token. To predict token t, the model must process all previous tokens $1, \dots, t-1$.
- Generating each new token requires loading the entire model's weights (gigabytes of parameters) from High Bandwidth Memory (HBM) to GPU SRAM.
- The GPU performs only a single token's matrix-vector multiplication (GEMV) before discarding the weights or loading the next layer.
- Because memory read speeds (HBM bandwidth) are orders of magnitude slower than arithmetic

processing speeds, the GPU Tensor Cores sit idle waiting for weights, making token decoding bound by memory bandwidth.

---

2. The Roofline Model & Arithmetic Intensity

The **Roofline Model** maps a system's execution performance limits against its **Arithmetic Intensity** (I), which represents the ratio of arithmetic work (FLOPs) performed per byte of data transferred:

$$I = \frac{\text{Work (FLOPs)}}{\text{Memory Traffic (Bytes)}}$$

Hardware performance is bounded by two physical constraints:
1. **Peak Compute Performance** (P_{peak}): The maximum floating-point operations the processor can run per second (FLOP/s).
2. **Peak Memory Bandwidth** (B_{peak}): The maximum rate at which the processor can read or write data to memory (Bytes/sec).

The attainable performance (P) is modeled as:

$$P = \min\left(P_{\text{peak}}, I \cdot B_{\text{peak}}\right)$$

The transition point where the bottleneck shifts from memory-bound to compute-bound is the **Hardware Ridge Point** (I_{ridge}):

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B_{\text{peak}}}$$

> 💡 **TIP:**
>
> **The Memory Bus Analogy**:
> Think of the GPU as a massive assembly line (the Tensor Cores) and VRAM (HBM) as the parts warehouse.
> - **In Prefill (Compute-Bound)**: A single massive pallet of materials (the model weights) is loaded onto the assembly line, and the workers spend a long time processing thousands of tasks (tokens) in parallel. The workers are kept fully saturated—this is compute-bound.
> - **In Decode (Memory-Bandwidth-Bound)**: The assembly line must load the entire blueprint catalogue (the whole model weights) from the warehouse just to stamp a single custom label (one new token), after which the catalog is discarded, and the next layer's catalog is loaded. The shipping trucks (memory bus) are bottlenecked, while the workers sit idle waiting for the next catalog—this is memory-bandwidth-bound.

Step-by-Step Hand Calculation on a 1B Parameter Model

Let's compute the arithmetic intensity for an LLM layer with $P = 1 \times 10^9$ (1 Billion) parameters executing on an NVIDIA A100 GPU (SXM4, 80GB VRAM) in FP16 precision:
- **NVIDIA A100 SXM4 peak FP16/BF16 compute** (P_{peak}): $312 \text{ TFLOP/s} = 312 \times 10^{12} \text{ FLOP/s}$.
- **NVIDIA A100 SXM4 peak memory bandwidth** (B_{peak}): $2.0 \text{ TB/s} = 2.0 \times 10^{12} \text{ Bytes/s}$.
- **Hardware Ridge Point** (I_{ridge}):

$$I_{\text{ridge}} = \frac{312 \times 10^{12}}{2.0 \times 10^{12}} = 156 \text{ FLOP/s per Byte}$$

- If a kernel's arithmetic intensity $I < 156$, it is **memory-bandwidth-bound**.
- If $I > 156$, it is **compute-bound**.

Scenario A: Prefill Phase (Batch Size $b = 1$, Prompt Length $N = 1024$ tokens)

During prefill, the model processes all 1024 prompt tokens in parallel.
1. **FLOPs Estimation**: A forward pass requires approximately $2 \times P$ operations per token:

$$\text{FLOPs} = 2 \times (1 \times 10^9) \times 1024 = 2.048 \times 10^{12} \text{ FLOPs} = 2.048 \text{ TFLOPs}$$

2. **Memory Traffic Estimation**: We load the model weights in FP16 ($2 \text{ bytes per parameter}$):

$$\text{Bytes} = P \times 2 = 1 \times 10^9 \times 2 = 2.0 \times 10^9 \text{ Bytes} = 2 \text{ GB}$$

3. **Arithmetic Intensity (I_{prefill})**:

$$I_{\text{prefill}} = \frac{2.048 \times 10^{12} \text{ FLOPs}}{2.0 \times 10^9 \text{ Bytes}} = 1024 \text{ FLOP/s per Byte}$$

4. **Attainable Performance**: Since $1024 > 156$ (well past the ridge point), the prefill phase is **compute-bound**.

Scenario B: Decode Phase (Batch Size $b = 1$, generating $N = 1$ new token)

During decoding, we process exactly 1 token at a time.
1. **FLOPs Estimation**:

$$\text{FLOPs} = 2 \times (1 \times 10^9) \times 1 = 2 \times 10^9 \text{ FLOPs} = 2 \text{ GFLOPs}$$

2. **Memory Traffic Estimation**: We must load the entire 2 GB model weights to calculate the output for this single token:

$$\text{Bytes} = P \times 2 = 2.0 \times 10^9 \text{ Bytes} = 2 \text{ GB}$$

3. **Arithmetic Intensity (I_{decode})**:

$$I_{\text{decode}} = \frac{2 \times 10^9 \text{ FLOPs}}{2.0 \times 10^9 \text{ Bytes}} = 1.0 \text{ FLOP/s per Byte}$$

4. **Attainable Performance**: Since $1.0 \ll 156$, the decode phase is **memory-bandwidth-bound**. The attainable performance is capped at:

$$P_{\text{decode}} = 1.0 \times (2.0 \times 10^{12} \text{ Bytes/s}) = 2.0 \text{ TFLOP/s}$$

Only 0.64% $(2/312)$ of the A100 GPU's raw compute capability is utilized!

---

3. Two-Phase Inference Lifecycle

LLM serving engines split generation into two sequential processing phases:

```
 <div style="display: flex; flex-direction: column; gap: 20px; font-family: 'Segoe UI', sans-serif; margin: 20px 0;">
  <!-- Prefill Phase Card -->
  <div style="border: 1px solid #3b82f6; border-radius: 8px; background-color: #f8fafc; padding: 16px; box-shadow: 0 4px 6px rgba(59, 130, 246, 0.05);">
    <div style="display: flex; align-items: center; gap: 10px; margin-bottom: 8px;">
      <span style="background-color: #3b82f6; color: white; padding: 4px 8px; border-radius: 4px; font-size: 11px; font-weight: bold; text-transform: uppercase;">Phase 1</span>
      <h4 style="margin: 0; font-size: 16px; color: #1e3a8a;">Prefill Phase (Prompt Processing)</h4>
    </div>
    <p style="margin: 0; font-size: 13px; color: #334155; line-height: 1.5;">
      Processes all prompt tokens simultaneously in a single forward pass. Computes and saves the initial key-value (KV) states into the KV Cache. Highly parallelized, maximizing tensor core utilization. Bounded by <strong>Compute (FLOP/s)</strong>.
    </p>
  </div>

  <!-- Arrow -->
```

```
  <div style="text-align: center; color: #64748b; font-size: 20px; font-weight:
bold;">↓</div>

  <!-- Decode Phase Card -->
  <div style="border: 1px solid #10b981; border-radius: 8px; background-color: #f8fafc;
padding: 16px; box-shadow: 0 4px 6px rgba(16, 185, 129, 0.05);">
    <div style="display: flex; align-items: center; gap: 10px; margin-bottom: 8px;">
      <span style="background-color: #10b981; color: white; padding: 4px 8px; border-
radius: 4px; font-size: 11px; font-weight: bold; text-transform: uppercase;">Phase
2</span>
      <h4 style="margin: 0; font-size: 16px; color: #065f46;">Decode Phase (Token
Generation)</h4>
    </div>
    <p style="margin: 0; font-size: 13px; color: #334155; line-height: 1.5;">
      Generates tokens one-by-one autoregressively. Each step takes the single newly
generated token, reads the prior KV Cache tokens from VRAM, performs a matrix-vector
product, and appends the new token's key-value states to the cache. Bounded by
<strong>Memory Bandwidth (GB/s)</strong>.
    </p>
  </div>
</div>
```

---

4. Prefill-Decode (PD) Disaggregation

The Problem

When prefill and decode phases run on the same GPU instance, they compete for execution slots.
- A compute-heavy prefill request arriving in the queue can block ongoing decode requests. This is called **Head-of-Line Blocking**.
- Because prefill and decode require different optimal batch sizes (prefill benefits from small batches to minimize latency; decode benefits from large batches to saturate memory bandwidth), running them on the same GPU compromises execution efficiency.

The Solution: PD Disaggregation

PD Disaggregation decouples the physical hardware nodes assigned to these phases:

```
 <div style="display: flex; justify-content: space-between; align-items: stretch; gap:
20px; font-family: 'Segoe UI', sans-serif; margin: 24px 0;">
  <!-- Prefill Node Pool -->
  <div style="flex: 1; border: 1.5px solid #2563eb; border-radius: 8px; background-
color: #f1f5f9; padding: 16px; text-align: center;">
    <div style="font-weight: 700; color: #1e3a8a; font-size: 14px; margin-bottom: 12px;
text-transform: uppercase; letter-spacing: 0.05em;">Prefill Nodes (GPUs)</div>
```

```
    <div style="font-size: 12px; color: #475569; margin-bottom: 8px;">Compute-optimized
batching</div>
    <div style="background-color: #dbeafe; color: #1e40af; font-size: 12px; padding:
6px; border-radius: 4px; font-weight: 600; margin-top: 4px; border: 1px dashed
#93c5fd;">High Tensor Core Saturation</div>
  </div>

  <!-- Network Interconnect (Arrow Bridge) -->
  <div style="display: flex; flex-direction: column; justify-content: center; align-
items: center; width: 120px; text-align: center;">
    <span style="font-size: 11px; font-weight: bold; color: #64748b; margin-bottom:
4px;">KV Cache Transfer</span>
    <div style="width: 100%; height: 6px; background: linear-gradient(90deg, #2563eb,
#10b981); border-radius: 3px;"></div>
    <span style="font-size: 10px; color: #94a3b8; margin-top: 4px; font-family:
monospace;">RDMA / NVLink</span>
  </div>

  <!-- Decode Node Pool -->
  <div style="flex: 1; border: 1.5px solid #10b981; border-radius: 8px; background-
color: #f1f5f9; padding: 16px; text-align: center;">
    <div style="font-weight: 700; color: #065f46; font-size: 14px; margin-bottom: 12px;
text-transform: uppercase; letter-spacing: 0.05em;">Decode Nodes (GPUs)</div>
    <div style="font-size: 12px; color: #475569; margin-bottom: 8px;">Memory-bandwidth
optimized batching</div>
    <div style="background-color: #d1fae5; color: #065f46; font-size: 12px; padding:
6px; border-radius: 4px; font-weight: 600; margin-top: 4px; border: 1px dashed
#6ee7b7;">Max Memory Bandwidth Saturation</div>
  </div>
</div>
```

1. **Prefill Nodes**: Process the prompt sequences and compute the initial KV states.
2. **KV Cache Transfer**: Send the generated KV states over a high-speed network (InfiniBand, RDMA, or NVLink) to the Decode nodes.
3. **Decode Nodes**: Append the states to memory and execute the low-intensity decode steps without blocking from new prompt arrivals.

Serving Topology Trade-offs (Unified vs. PD Disaggregated)

Topology	Pros	Cons	Production Use Case
Unified serving (Single GPU / Node)	• Simple orchestration and zero network overhead. • No need for expensive high-speed RDMA interconnects. • Easy deployment on standard cloud instances.	• Heavy Head-of-Line blocking (a long prefill prompt pauses decodes). • Poor GPU resource utilization due to mixed compute/memory bounds.	Low-to-medium volume deployments, personal assistants, or offline jobs.
PD Disaggregation (Split Pool serving)	• Bypasses Head-of-Line blocking entirely. • Allows compute-optimized nodes (H100) for prefill, and memory-bandwidth nodes (A100) for decode. • Distinct queue batching policies.	• Heavy KV Cache transfer overhead across network. • High infrastructure complexity and cost (requires RDMA/InfiniBand). • Routing layer overhead.	High-scale LLM API providers (OpenAI, Anyscale) serving millions of concurrent requests.

---

5. Core Latency & Throughput Metrics

Production LLM hosting targets three metrics to satisfy client SLAs:

Metric	Definition	Critical For	Hard Limits
TTFT (Time to First Token)	Time elapsed from sending a request to receiving the first generated token.	Interactive Chat, Real-Time Interfaces.	Bounded by prefill execution speed and request queue delays.
TPOT / ITL (Time Per Output Token)	The average time interval between generating successive tokens.	Reading comfort, downstream streaming consumers.	Bounded by GPU HBM memory bandwidth.
TPS (Tokens Per Second)	Total output tokens generated per second across all users ($TPS = \text{Batch Size} \times \frac{1}{\text{TPOT}}$).	Offline batch processing pipelines (summarization, indexing).	Bounded by peak scheduling efficiency and VRAM capacity.

Interview Questions & Production Trade-offs

- What problem does this solve?
- Why was it introduced?
- What are its limitations?
- Computational Complexity (Time & Memory)
- Component Variable Denotation Legend (Explicitly defining $N, L, |V|, d, m, K, T, C, P$)
- Production Use Cases
- Follow-up questions interviewers ask

02. Decoding Strategies & Structured Generation: Sampling Mechanics and Constraints

An LLM outputs raw unbounded scores (logits) for every token in its vocabulary. The decoding strategy controls how these logits are processed to select the next token. Selecting the correct sampling parameters and constraints determines whether an LLM produces factual code, cohesive conversation, or structured JSON payloads.

---

1. Deterministic Decoding

Deterministic strategies always select the token path with the highest probability, producing predictable, reproducible outputs.

Greedy Decoding

At each generation step t, greedy decoding selects the token index x_t that has the highest individual probability score:

$$x_t = \arg\max_i P(\text{token}_i \mid S_{t-1})$$

- **Intuition**: "Short-sighted" decision making. It selects the immediately best-looking token, ignoring the downstream sequence impact.
- **Pros**:
- Zero computational overhead (just an $O(1)$ argmax).
- Perfect for deterministic, factual tasks like mathematical calculations and code generation.
- **Cons**:
- Frequently gets trapped in infinite repetitive loops (e.g., "and then and then and then...").
- Misses paths where selecting a lower-probability token now leads to a globally higher-probability sentence.

Beam Search

Instead of keeping only the single best token, Beam Search maintains a fixed number of candidate sequences (the *beam width* B). At each step, it expands all B paths, computes their cumulative log-probabilities, and keeps the top B sequences:

$$\text{Score}(X_{1:t}) = \sum_{\tau=1}^{t} \log P(x_\tau \mid X_{1:\tau-1})$$

To prevent favoring shorter paths, a **Length Penalty** (L_p) is applied:

$$\text{Score}_{\text{normalized}}(X_{1:t}) = \frac{\sum_{\tau=1}^{t} \log P(x_\tau \mid X_{1:\tau-1})}{\left(\frac{5+t}{6}\right)^{\alpha}}$$

where α is the length penalty coefficient (typically 0.5 to 1.0).
- **Intuition**: Explores a broader search space, keeping alternative paths open to find a globally optimal sequence.
- **Pros**:
- Highly effective for translation and summarization tasks where global output structure is critical.
- **Cons**:
- **Memory Killer (VRAM)**: Keeping B active sequence paths requires storing B copies of the KV Cache for that request. This kills GPU concurrency, making Beam Search memory-prohibitive for high-throughput production gateways. It is rarely supported or enabled in high-concurrency engines like vLLM.

---

2. Stochastic Sampling & Temperature Math

Stochastic decoding introduces randomness by sampling from the probability distribution.

Temperature Scaling

Before applying the Softmax function to convert raw logits z_i into probabilities P_i, logits are divided by a temperature parameter $T \in (0, \infty)$:

$$P_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

Step-by-Step Hand Calculation on a 3-Token Vocabulary

Let our raw logit vector be $z = [2.0, 1.0, 0.0]$ for vocabulary tokens $[\text{"cat"}, \text{"dog"}, \text{"fish"}]$.

1. Reference Temperature ($T = 1.0$)

- Scale logits: $z/1.0 = [2.0, 1.0, 0.0]$
- Exponentiate: $e^{2.0} \approx 7.389$, $e^{1.0} \approx 2.718$, $e^{0.0} = 1.0$
- Sum of Exponents: $7.389 + 2.718 + 1.0 = 11.107$
- Probabilities:
- $P(\text{"cat"}) = 7.389/11.107 \approx 0.665$
- $P(\text{"dog"}) = 2.718/11.107 \approx 0.245$
- $P(\text{"fish"}) = 1.0/11.107 \approx 0.090$

2. Low Temperature ($T = 0.5$ - Sharpening the distribution)

- Scale logits: $z/0.5 = [4.0, 2.0, 0.0]$
- Exponentiate: $e^{4.0} \approx 54.598$, $e^{2.0} \approx 7.389$, $e^{0.0} = 1.0$
- Sum of Exponents: $54.598 + 7.389 + 1.0 = 62.987$
- Probabilities:
- $P(\text{"cat"}) = 54.598/62.987 \approx 0.867$
- $P(\text{"dog"}) = 7.389/62.987 \approx 0.117$
- $P(\text{"fish"}) = 1.0/62.987 \approx 0.016$
- *Effect: The probability of the top token increases from 66.5% to 86.7%. As $T \to 0$, sampling converges to greedy decoding.*

3. High Temperature ($T = 2.0$ - Flattening the distribution)

- Scale logits: $z/2.0 = [1.0, 0.5, 0.0]$
- Exponentiate: $e^{1.0} \approx 2.718$, $e^{0.5} \approx 1.649$, $e^{0.0} = 1.0$
- Sum of Exponents: $2.718 + 1.649 + 1.0 = 5.367$
- Probabilities:
- $P(\text{"cat"}) = 2.718/5.367 \approx 0.506$
- $P(\text{"dog"}) = 1.649/5.367 \approx 0.307$
- $P(\text{"fish"}) = 1.0/5.367 \approx 0.186$
- *Effect: The probability distribution becomes flatter and more uniform. As $T \to \infty$, sampling approaches a uniform random distribution.*

---

3. Top-K, Top-p, and Min-p Truncations

To prevent sampling highly improbable "garbage" tokens (the tail of the distribution), we truncate the vocabulary candidate list before sampling.

Strategy	Truncation Logic	Mathematical Formulation	Pros & Cons	Production Insight
Top-K	Keeps only the K tokens with the highest individual logits.	$\lvert V_{\text{candidate}} \rvert = K$	• **Pros**: Simple, low CPU sorting overhead. • **Cons**: Rigid; includes garbage tail tokens when the distribution is sharp, or chops off valid candidates when flat.	Older baseline; rarely used on its own in modern production gateways.
Top-p (Nucleus)	Keeps the smallest subset of tokens whose cumulative probability exceeds p.	$\sum_{i \in V_{\text{candidate}}} P_i \geq p$	• **Pros**: Dynamic width adjusts to model confidence. • **Cons**: If the top token has 98% probability, a $p = 0.99$ threshold still forces the inclusion of garbage tail tokens.	Current industry standard for conversational chat (e.g. OpenAI default values).
Min-p	Truncates tokens whose probability is below a fraction of the top token's probability $p_{\max}$.	$P_i < p_{\max} \times p_{\text{threshold}}$	• **Pros**: Highly dynamic scaling; discards tail tokens in confident states but preserves choices in ambiguous states. • **Cons**: Tuning $p_{\text{threshold}}$ requires intuition.	Modern SOTA replacement for Top-p; yields cleaner, more robust generation outputs.

---

4. Repetition Penalties

To prevent semantic loops, logits are modified based on token presence in the history prompt H:
- **Frequency Penalty**: Penalizes tokens based on how many times they have already appeared in the output history.

- **Presence Penalty**: Applies a constant penalty to any token that has appeared at least once in the history.
- **Repetition Penalty (Llama Style)**: Multiplies logits directly:

$$z_i = \begin{cases} z_i / \theta & \text{if } z_i \geq 0 \\ z_i \cdot \theta & \text{if } z_i < 0 \end{cases}$$

where $\theta > 1.0$ reduces the logit score of active tokens.

---

5. Constrained Decoding & Structured Generation

Generating structured formats like JSON or SQL requires restricting token selection at each step to match a target schema.

Regex & CFG Logit Masking

During decoding, standard engines (such as **Outlines** or **XGrammar**) build a finite state machine (FSM) or pushdown automaton from the target regex/grammar.
1. At step t, the FSM determines the set of all vocabulary token strings that are valid next transitions.
2. The engine creates a binary logit mask M, setting the score of invalid tokens to $-\infty$:

$$z_i = z_i + M_i, \quad M_i = \begin{cases} 0 & \text{valid} \\ -\infty & \text{invalid} \end{cases}$$

3. Softmaxing ensures that invalid tokens have exactly 0% probability of being sampled.

Mitigating Parser Overhead

Historically, calculating the logit mask at each step introduced significant CPU latency. **XGrammar** resolves this by pre-computing state transition trees in C++ and caching token masks, allowing grammar-constrained serving to run at near-native speeds without hurting TPOT.

---

6. Decoding Strategy Decision Matrix

Task Domain	Optimal Temperature	Truncation parameters	Penalties	Strategy
Code / Math	0.0 (Greedy)	N/A	None	Greedy Decoding
JSON Extraction	0.0	N/A	None	Outlines Grammar constraint
Conversational Chat	0.7	Top-p = 0.90 or Min-p = 0.05	Repetition = 1.05	Stochastic Sampling
Creative Writing	1.2	Min-p = 0.10	Presence = 0.10	High Temp Stochastic

---

Interview Questions & Production Trade-offs

- What problem does this solve?
- Why was it introduced?
- What are its limitations?
- Computational Complexity (Time & Memory)
- Component Variable Denotation Legend (Explicitly defining $N, L, |V|, d, m, K, T, C, P$)
- Production Use Cases
- Follow-up questions interviewers ask

03. KV Cache & Attention Mechanics: Memory Bottlenecks and Virtualization

Autoregressive transformer inference requires storing the Key (K) and Value (V) state projections of all past context tokens. This mechanism—the **KV Cache**—saves the system from recomputing self-attention over historical tokens at every single generation step, converting a quadratic time complexity $O(N^2)$ attention pass into an efficient $O(1)$ query projection. However, caching historical states shifts the primary system bottleneck from processing computation directly to managing VRAM memory footprints.

---

1. The KV Cache Bottleneck

In a standard Multi-Head Attention (MHA) block, when generating token t, the query vector q_t must be multiplied by keys from all previous tokens:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_{\text{head}}}}\right)V$$

Without caching, to generate token t, we would have to project the key and value vectors for all past $t-1$ tokens from scratch. This would require $O(t^2)$ projection operations over the sequence. By caching past key and value vectors in memory, at step t we only calculate q_t, k_t, v_t for the single new token, load the historical keys $K_{1:t-1}$ and values $V_{1:t-1}$ from VRAM, perform attention, and write the new k_t, v_t to the cache.
While this reduces FLOPs, it creates a massive VRAM footprint that scales linearly with sequence length (l) and batch size (b).

---

2. Exact KV Cache Memory Formula

The exact VRAM footprint (in Bytes) required to store the KV Cache across an entire LLM inference pass is given by:

$$\text{VRAM}_{\text{KV}} = 2 \times b \times l \times s \times n_{\text{heads}} \times d_{\text{head}} \times \text{bytes\_per\_element}$$

Where:
- b: Batch size.

- l: Sequence context length (in tokens).
- s: Number of layers in the model.
- n_{heads}: Number of key-value attention heads per layer (varies based on attention architecture).
- d_{head}: Dimensionality of each attention head.
- bytes_per_element: Data format size (2 bytes for FP16/BF16, 1 byte for FP8/INT8).
- The leading factor of 2 accounts for storing *both* Key (K) and Value (V) tensors.

Step-by-Step Hand Calculations for Llama-3 Models

Let's calculate the required VRAM for Llama-3 8B and Llama-3 70B models at batch size $b = 16$ using FP16 precision ($\text{bytes\_per\_element} = 2$) across varying context windows.

Specifications Table

Model	Layers (s)	KV Heads (n_{heads})	Head Dim (d_{head})
Llama-3 8B	32	8 (Grouped-Query Attention)	128
Llama-3 70B	80	8 (Grouped-Query Attention)	128

---

1. Llama-3 8B (at $b = 16$)

Case A: Context Length $l = 32\text{k}$ ($32,768$ tokens)

- Formula substitution:

$$\text{VRAM}_{\text{KV}} = 2 \times 16 \times 32768 \times 32 \times 8 \times 128 \times 2 \text{ bytes}$$

- Calculate step-by-step:
- $2 \times 16 \times 32768 = 1,048,576$ (tokens across batch)
- $1,048,576 \times 32 = 33,554,432$ (layer-tokens)
- $33,554,432 \times 8 = 268,435,456$ (active heads)
- $268,435,456 \times 128 = 34,359,738,368$ (elements)
- $34,359,738,368 \times 2 \text{ bytes} = 68,719,476,736 \text{ bytes}$
- Convert to GiB:

$$\text{GiB} = \frac{68,719,476,736}{1024^3} = 64.00 \text{ GiB}$$

Case B: Context Length $l = 128\text{k}$ ($131,072$ tokens)

- Calculate:

$$\text{VRAM}_{\text{KV}} = 2 \times 16 \times 131072 \times 32 \times 8 \times 128 \times 2 = 274,877,906,944 \text{ bytes}$$

- Convert to GiB:

$$\text{GiB} = \frac{274,877,906,944}{1024^3} = 256.00 \text{ GiB}$$

---

2. Llama-3 70B (at $b = 16$)

Case A: Context Length $l = 32\text{k}$ ($32,768$ tokens)

- Formula substitution:

$$\text{VRAM}_{\text{KV}} = 2 \times 16 \times 32768 \times 80 \times 8 \times 128 \times 2 \text{ bytes}$$

- Calculate step-by-step:
- $2 \times 16 \times 32768 = 1,048,576$ (tokens across batch)
- $1,048,576 \times 80 = 83,886,080$ (layer-tokens)
- $83,886,080 \times 8 = 671,088,640$ (active heads)
- $671,088,640 \times 128 = 85,899,345,920$ (elements)
- $85,899,345,920 \times 2 \text{ bytes} = 171,798,691,840 \text{ bytes}$
- Convert to GiB:

$$\text{GiB} = \frac{171,798,691,840}{1024^3} = 160.00 \text{ GiB}$$

Case B: Context Length $l = 128\text{k}$ ($131,072$ tokens)

- Calculate:

$$\text{VRAM}_{\text{KV}} = 2 \times 16 \times 131072 \times 80 \times 8 \times 128 \times 2 = 687,194,767,360 \text{ bytes}$$

- Convert to GiB:

$$\text{GiB} = \frac{687,194,767,360}{1024^3} = 640.00 \text{ GiB}$$

---

3. Attention Variants for Memory Reduction

To mitigate the VRAM demand of the KV Cache, modern LLM architectures adjust the ratio of query heads to KV heads:

```
 <div style="display: flex; flex-direction: column; gap: 16px; font-family: 'Segoe UI', sans-serif; margin: 20px 0;">
  <!-- MHA -->
  <div style="border: 1px solid #cbd5e1; border-radius: 6px; padding: 12px; background-color: #f8fafc;">
    <h4 style="margin: 0 0 6px 0; color: #0f172a; font-size: 14px;">1. Multi-Head Attention (MHA)</h4>
    <div style="font-size: 12px; color: #475569; margin-bottom: 4px;">Each Query head has a dedicated Key and Value head.</div>
    <div style="font-family: monospace; font-size: 11px; color: #e11d48; font-weight: bold;">KV Heads : Query Heads = 1 : 1</div>
  </div>

  <!-- MQA -->
  <div style="border: 1px solid #cbd5e1; border-radius: 6px; padding: 12px; background-color: #f8fafc;">
    <h4 style="margin: 0 0 6px 0; color: #0f172a; font-size: 14px;">2. Multi-Query Attention (MQA)</h4>
    <div style="font-size: 12px; color: #475569; margin-bottom: 4px;">All Query heads share a single Key and Value head. Reduces KV Cache VRAM footprint significantly but degrades capability.</div>
    <div style="font-family: monospace; font-size: 11px; color: #e11d48; font-weight: bold;">KV Heads : Query Heads = 1 : H</div>
  </div>

  <!-- GQA -->
  <div style="border: 1px solid #cbd5e1; border-radius: 6px; padding: 12px; background-color: #f8fafc;">
    <h4 style="margin: 0 0 6px 0; color: #0f172a; font-size: 14px;">3. Grouped-Query Attention (GQA)</h4>
    <div style="font-size: 12px; color: #475569; margin-bottom: 4px;">Query heads are grouped; each group shares a single Key and Value head. Combines the speed of MQA with the capacity of MHA.</div>
    <div style="font-family: monospace; font-size: 11px; color: #e11d48; font-weight: bold;">KV Heads : Query Heads = 1 : G (Typically 1 : 8 or 1 : 4)</div>
  </div>
</div>
```

Attention Variant Trade-offs

Variant	KV : Query Heads	Pros	Cons	Production Choice
Multi-Head Attention (MHA)	$1:1$	• Maximum representational capacity (each query head has private KV context).	• Maximum VRAM footprint; severely limits batch sizes and sequence length.	Older baseline models (e.g., LLaMA-1, GPT-3).
Multi-Query Attention (MQA)	$1:H$	• Minimal KV Cache VRAM footprint (up to 8x-32x footprint reduction).	• Degrades model capacity; performance drops on long, complex documents.	Used in specialized low-resource models (e.g., Falcon).
Grouped-Query Attention (GQA)	$1:G$ (e.g., $1:8$)	• Combines MHA's high quality with MQA's low memory consumption.	• Marginally more complex query-to-group mapping logic in code.	Standard for modern SOTA models (LLaMA-3, Mistral, Command-R).

- **VRAM Saving**: If a model uses a Grouped-Query Attention ratio of $8:1$ (e.g. Llama-3), its KV Cache VRAM footprint is reduced by **8x** compared to standard MHA.

---

4. PagedAttention & Virtual Memory Management

The Problem: Memory Fragmentation

Traditional serving systems allocated KV Cache memory for each request as a contiguous chunk of VRAM sized to the *maximum* possible sequence length (e.g. 4096 tokens).
- **Internal Fragmentation**: If a request terminated early after generating 10 tokens, the remaining pre-allocated memory blocks remained reserved and unused.
- **External Fragmentation**: Memory allocations of varying sizes created gaps of un-allocatable space, leading to CUDA Out-Of-Memory errors despite having free memory bytes.
This wasted up to $60\% - 80\%$ of available GPU memory.

The Solution: PagedAttention (vLLM)

PagedAttention borrows the concept of virtual memory and paging from operating systems:
1. **Physical Blocks**: The GPU's VRAM is divided into non-contiguous physical blocks of a fixed size (e.g. 16 tokens).

2. **Logical Blocks**: The serving engine maps a request's logical sequence of tokens to these physical blocks.
3. **Block Table**: A dynamic lookup table tracks which logical blocks map to which physical memory blocks.
4. **On-Demand Allocation**: As new tokens are generated, the engine allocates new blocks dynamically. Blocks do not need to be contiguous in VRAM, eliminating fragmentation and allowing VRAM utilization to approach 96%.

---

5. Prefix Caching vs. SGLang RadixAttention

For multi-turn conversations, agent execution steps, and RAG applications, successive queries share identical system prompts or reference documents. Caching these shared prefixes prevents recomputing their KV states.

vLLM Block-level Hash Caching

vLLM implements Automatic Prefix Caching (APC) by computing a cryptographic hash of the token IDs inside a block. If a new request's starting blocks match the hash of cached blocks, the engine reuses the physical blocks directly.
- **Limitation**: The prefix matching is rigid and block-grained. It cannot handle dynamic, arbitrary prefix paths or branching chats efficiently.

SGLang RadixAttention

SGLang models the KV Cache as a dynamic **Radix Tree** data structure:
- Prompt prefixes (e.g. System Prompt, Document Context, Chat Turn 1) are stored as nodes in a tree.
- When a new request arrives, SGLang traverses the tree to find the longest matching prefix path.
- The matching KV cache is reused, and the new execution branches off as a child node.
- If the GPU memory fills up, SGLang uses a Least Recently Used (LRU) eviction policy to evict child leaves while keeping the common root prefixes cached.
This accelerates TTFT in agent loops by $2\times - 5\times$ compared to standard block hashing.

---

6. FlashAttention Hardware Acceleration

FlashAttention (1, 2, and 3) optimizes attention compute speed, but it is important to understand its hardware limits:
- **How it works**: Standard attention writes intermediate QK^T matrices to slow High Bandwidth Memory (HBM). FlashAttention uses *tiling* and *online softmax* to load blocks of queries, keys, and

values into fast GPU SRAM, perform computations, and write only the final outputs back to HBM.
- **VRAM Impact**: FlashAttention reduces the **intermediate activation memory** during the forward pass. However, it does *not* reduce the VRAM footprint of the stored KV Cache in VRAM. The stored key-value states must still occupy the exact VRAM size derived by the memory formula.

Interview Questions & Production Trade-offs

- What problem does this solve?
- Why was it introduced?
- What are its limitations?
- Computational Complexity (Time & Memory)
- Component Variable Denotation Legend (Explicitly defining $N, L, |V|, d, m, K, T, C, P$)
- Production Use Cases
- Follow-up questions interviewers ask

04. Model Quantization & Compression: Precision and Conversion Mechanics

LLM serving requires significant VRAM memory. To reduce costs and allow large models to run on standard hardware, we utilize **Model Quantization** to reduce the bit-width of weights and activations (e.g. from 16-bit floating point down to 8-bit or 4-bit integers), dramatically lowering VRAM usage and boosting memory-bound decoding speeds.

---

1. Floating Point Formats & Data Precision

Modern GPUs process numbers in several precision formats:
- **FP32 (Single Precision)**: 32 bits. 1 sign bit, 8 exponent bits (range), 23 fraction/mantissa bits (precision). Standard for training base calculations, but too memory-intensive for online serving.
- **FP16 (Half Precision)**: 16 bits. 1 sign, 5 exponent, 10 fraction. Standard model weights format.
- **BF16 (Brain Floating Point)**: 16 bits. 1 sign, 8 exponent, 7 fraction. Matches FP32's dynamic range but with reduced numerical precision. Prevents underflow/overflow during fine-tuning.
- **FP8 (8-Bit Float)**: Used in NVIDIA Hopper Tensor Cores:
- **E4M3**: 1 sign, 4 exponent, 3 mantissa. Maximizes precision; preferred for activation layers during forward inference steps.
- **E5M2**: 1 sign, 5 exponent, 2 mantissa. Matches FP16's dynamic range; preferred for weight storage and gradients.

---

2. Quantization Mechanics: Symmetric vs. Asymmetric

Quantization maps a continuous range of real-world values $r \in [\min(X), \max(X)]$ to a discrete integer grid $q \in [q_{\min}, q_{\max}]$ (such as $[-128, 127]$ for signed INT8).

Symmetric Quantization

Maps the real range symmetrically around zero ($z = 0$).
- **Scale Factor (s)**:

$$s = \frac{\max(|X|)}{q_{\max}}$$

- **Quantization**:

$$q = \text{clip}\left(\text{round}\left(\frac{X}{s}\right), q_{\min}, q_{\max}\right)$$

- **De-quantization**:

$$r \approx s \cdot q$$

Asymmetric Quantization

Maps the real minimum and maximum values exactly to the integer minimum and maximum boundaries, offset by a **Zero-Point** (z).

- **Scale Factor (s)**:

$$s = \frac{\max(X) - \min(X)}{q_{\max} - q_{\min}}$$

- **Zero-Point (z)**:

$$z = \text{round}\left(\frac{-\min(X)}{s}\right) + q_{\min}$$

- **Quantization**:

$$q = \text{clip}\left(\text{round}\left(\frac{X}{s}\right) + z, q_{\min}, q_{\max}\right)$$

- **De-quantization**:

$$r \approx s \cdot (q - z)$$

Step-by-Step Hand Calculation: Asymmetric INT8 Quantization

Let our real-valued activation vector be:

$$X = [-1.5, 0.5, 2.0]$$

We want to quantize X to signed INT8, where $q_{\min} = -128$ and $q_{\max} = 127$.

Step 1: Identify bounds

- $\min(X) = -1.5$
- $\max(X) = 2.0$

Step 2: Compute Scale (s)

$$s = \frac{\max(X) - \min(X)}{q_{\max} - q_{\min}} = \frac{2.0 - (-1.5)}{127 - (-128)} = \frac{3.5}{255} \approx 0.0137255$$

Step 3: Compute Zero-Point (z)

$$z = \text{round}\left(\frac{-(-1.5)}{0.0137255}\right) + (-128) = \text{round}(109.28) - 128 = 109 - 128 = -19$$

Step 4: Quantize values $q = \text{round}(X/s) + z$

1. **For $X_1 = -1.5$:**

$$q_1 = \text{round}\left(\frac{-1.5}{0.0137255}\right) - 19 = \text{round}(-109.28) - 19 = -109 - 19 = -128$$

2. **For $X_2 = 0.5$:**

$$q_2 = \text{round}\left(\frac{0.5}{0.0137255}\right) - 19 = \text{round}(36.43) - 19 = 36 - 19 = 17$$

3. **For $X_3 = 2.0$:**

$$q_3 = \text{round}\left(\frac{2.0}{0.0137255}\right) - 19 = \text{round}(145.71) - 19 = 146 - 19 = 127$$

Quantized vector: $q = [-128, 17, 127]$

Step 5: De-quantize values $r \approx s \cdot (q - z)$

1. $r_1 \approx 0.0137255 \times (-128 - (-19)) = 0.0137255 \times (-109) = -1.496 \approx -1.5$
2. $r_2 \approx 0.0137255 \times (17 - (-19)) = 0.0137255 \times 36 = 0.494 \approx 0.5$
3. $r_3 \approx 0.0137255 \times (127 - (-19)) = 0.0137255 \times 146 = 2.004 \approx 2.0$

---

3. Post-Training Quantization (PTQ) Paradigms

PTQ converts pre-trained models without retraining. We categorize PTQ methods by whether they target weights only or weight-activations.

Weight-Only Quantization

Quantizes weights to 4-bit, keeping activations in 16-bit. Weights are de-quantized to FP16 in SRAM before execution.
- **GPTQ**: Uses a second-order optimization method. It updates remaining weights to compensate for the error introduced by quantizing a specific column weight, using the inverse Hessian matrix.
- **AWQ (Activation-aware Weight Quantization)**: Observes that not all weights are equally important. Only 1% of weights (those corresponding to features with high activation magnitude) are salient. AWQ protects these channels from quantization error by applying a scaling factor rather than running expensive optimization passes.

Weight-Activation Quantization

Quantizes both weights and activations to INT8, allowing the model to utilize integer compute engines (INT8 Tensor Cores) directly.
- **SmoothQuant**: Emergent outlier features in activations (spikes up to 100x larger than median values) make activations difficult to quantize to INT8. SmoothQuant mathematically migrates this quantization difficulty from the activations (X) to the weights (W) by applying a scaling factor s:

$$Y = (X \cdot \text{diag}(s)^{-1}) \cdot (\text{diag}(s) \cdot W)$$

PTQ Algorithm Comparison Matrix

Algorithm	Quantization Scope	Pros	Cons	Production Choice
GPTQ	Weight-only (INT4/INT3)	• High quality compression at extreme bit-widths.	• High calibration computation overhead (requires inverse Hessian solvers).	Static weight-only deployment for memory reduction.

Algorithm	Quantization Scope	Pros	Cons	Production Choice
AWQ	Weight-only (INT4)	• Preserves outlier weights dynamically; yields higher accuracy than GPTQ. • Does not require complex Hessian approximations.	• Relies on specialized de-quantization kernels in SRAM at execution time.	Modern standard for INT4 serving on vLLM and SGLang.
SmoothQuant	Weight + Activation (W8A8)	• Enables native INT8 GEMM on Tensor Cores, yielding compute speedups.	• Higher perplexity degradation on very large models with massive activation outliers.	Enterprise serving where both latency and throughput must scale.

---

4. K-Quantization & GGUF (llama.cpp)

For edge servers and CPU execution, `llama.cpp` uses **Block-wise Quantization** formats (GGUF):
- Instead of scaling the entire layer, GGUF groups parameters into small blocks (e.g. 32 or 256 weights).
- Each block has a local scale factor and zero-point. This local grouping prevents outliers from degrading accuracy, allowing models like Llama-3 8B to run in 4-bit on standard laptops with minimal Perplexity loss.

---

5. Accuracy vs. VRAM Trade-Off Matrix

The table below summarizes the trade-offs of quantizing a Llama-3 8B model:

Precision Format	Weights VRAM	Perplexity Degradation	Optimal Hardware
BF16 / FP16	$16\ \text{GB}$	Baseline (0.0 change)	All GPUs
FP8 (E4M3/E5M2)	$8\ \text{GB}$	Negligible (< 0.02)	H100, B200
INT8 (SmoothQuant)	$8\ \text{GB}$	Low (< 0.05)	A100, T4, L4
INT4 (AWQ)	$4\ \text{GB}$	Minor (< 0.15)	Latency-bound GPUs
INT4 (GPTQ)	$4\ \text{GB}$	Minor (< 0.18)	Batch-bound GPUs

Interview Questions & Production Trade-offs

- What problem does this solve?
- Why was it introduced?
- What are its limitations?
- Computational Complexity (Time & Memory)
- Component Variable Denotation Legend (Explicitly defining $N, L, |V|, d, m, K, T, C, P$)
- Production Use Cases
- Follow-up questions interviewers ask

05. Batching, Scheduling & Serving: Iteration Scheduling and Parallelism

LLM hosting engines must manage multiple concurrent client requests. Because generating tokens autoregressively is a memory-bound process, serving requests sequentially results in low GPU utility. To maximize throughput and satisfy strict Service Level Agreements (SLAs), engines use advanced batching schedulers and multi-GPU tensor scaling.

---

1. Batching Evolution

Batching multiple requests together allows the GPU to share loaded model weights across multiple input tokens, increasing arithmetic intensity and Tensor Core utilization.

Static Batching

In static batching, requests are batched together and padded with dummy tokens to match the length of the longest request in the batch:
- **Wasted FLOPs**: The GPU executes computation on padding tokens, consuming memory and cycles.
- **Wasted VRAM**: Padding tokens consume block allocations in the KV Cache.
- **Easiest Finish Bottleneck**: The batch cannot release resources until the longest request completes, keeping shorter requests locked.

Dynamic Batching

Dynamic batching groups incoming requests into batches when they enter a queue buffer within a fixed time window. While it improves alignment, it still suffers from padding issues and head-of-line blocking under varying sequence lengths.

Continuous Batching (Iteration-level Scheduling)

Continuous batching operates at the iteration level rather than the sequence level:
1. The batch is stepped token-by-token.
2. At the end of a single decoding iteration, the scheduler inspects the queue.
3. Completed requests (those that hit `[EOS]` or max tokens) are immediately evicted from the active batch.

4. New requests (waiting in the queue) are inserted into the vacant slots.
This eliminates padding overheads, boosting throughput by $2\times - 4\times$ compared to static batching.

---

2. Request Scheduling Algorithms

Schedulers manage the allocation of physical blocks for the KV Cache:
- **First-Come-First-Served (FCFS)**: Default queue. Requests are processed in order of arrival. Can lead to preemption if KV Cache memory fills up.
- **Chunked Prefill**: Long prompt prefills require high compute time, which stalls ongoing decode requests and causes Inter-Token Latency (ITL) spikes. Chunked prefill splits a long prefill prompt into chunks (e.g. 512 tokens). It schedules one chunk of prefill alongside active decode requests in a single batch iteration, stabilizing ITL.

---

3. Multi-GPU Parallelism Strategies

When a model is too large to fit in the VRAM of a single GPU, or when we need to distribute the memory footprint, we apply parallelism:

Tensor Parallelism (TP - Megatron-LM)

Splits individual weight matrices across multiple GPUs within a single node. This requires fast GPU-to-GPU communication (e.g. NVLink) because synchronization is required at every layer.

```
 <div style="display: flex; flex-direction: column; gap: 20px; font-family: 'Segoe UI',
sans-serif; margin: 24px 0;">
  <!-- Column Parallel -->
  <div style="border: 1px solid #3b82f6; border-radius: 8px; background-color: #f8fafc;
padding: 16px;">
    <div style="font-weight: 700; color: #1e3a8a; font-size: 14px; margin-bottom: 8px;
text-transform: uppercase;">1. Column-Parallel Linear Layer</div>
    <div style="font-size: 13px; color: #334155; margin-bottom: 8px;">
      Weights $W$ are split vertically: $W = [W_1 \mid W_2]$. Input $X$ is replicated
on all GPUs.
    </div>
    <div style="background-color: #f1f5f9; padding: 10px; border-radius: 6px; font-
family: monospace; font-size: 12px; color: #0f172a; border-left: 3px solid #3b82f6;">
      GPU 1: Y1 = X * W1 <br>
      GPU 2: Y2 = X * W2 <br>
      Output: Y = Concatenate(Y1, Y2) [Requires All-Gather synchronization]
    </div>
  </div>
```

```
  <!-- Row Parallel -->
  <div style="border: 1px solid #10b981; border-radius: 8px; background-color: #f8fafc;
padding: 16px;">
    <div style="font-weight: 700; color: #065f46; font-size: 14px; margin-bottom: 8px;
text-transform: uppercase;">2. Row-Parallel Linear Layer</div>
    <div style="font-size: 13px; color: #334155; margin-bottom: 8px;">
      Weights $W$ are split horizontally: $W = [W_1^T, W_2^T]^T$. Input $X$ is split
column-wise: $X = [X_1 \mid X_2]$.
    </div>
    <div style="background-color: #f1f5f9; padding: 10px; border-radius: 6px; font-
family: monospace; font-size: 12px; color: #0f172a; border-left: 3px solid #10b981;">
      GPU 1: Y1 = X1 * W1 <br>
      GPU 2: Y2 = X2 * W2 <br>
      Output: Y = Y1 + Y2 [Requires All-Reduce Sum synchronization]
    </div>
  </div>
</div>
```

In a standard Transformer block, we chain these layers:
- Attention input projections (Q, K, V) are Column-Parallel.
- Attention output projection (O) is Row-Parallel.
- MLP Gate/Up projections are Column-Parallel.
- MLP Down projection is Row-Parallel.
This arrangement allows us to execute the entire attention block with only one `All-Reduce` operation at the end of attention and one `All-Reduce` at the end of MLP, minimizing communication overhead.

Pipeline Parallelism (PP)

Splits model layers across nodes (e.g. GPU 0 holds layers 1-10, GPU 1 holds 11-20).
- **Communication**: Communication is sequential, only passing activations at the boundaries of the pipeline stages.
- **Pipeline Bubble**: Idle wait times occur while nodes wait for activations from previous stages. This is mitigated by dividing batches into micro-batches (e.g., using 1F1B scheduling).

Context Parallelism (CP)

Splits the sequence length dimension of a long context input across GPUs. Each GPU processes a chunk of the tokens and computes local attention queries, keys, and values, exchanging attention metrics via ring-attention rings. Useful for sequences exceeding 100k tokens.

Parallelism Strategy Comparison Matrix

Strategy	Partition Dimension	Pros	Cons	Production Choice
Tensor Parallelism (TP)	Layer Weights (Columns/Rows)	• Lowest latency; highly efficient weight reuse. • Replicates execution states cleanly.	• Requires ultra-fast interconnects (NVLink); typically limited to 8 GPUs within a single node.	Scaling 70B class models within a single HGX node.
Pipeline Parallelism (PP)	Model Layers (Depth-wise)	• Scales across multiple server boxes over standard networking. • Low bandwidth demands.	• Introduces pipeline bubbles (nodes waiting for boundary activations). • High scheduling complexity (1F1B loops).	Scaling 405B class models across multiple interconnected nodes.
Context Parallelism (CP)	Sequence Length (Tokens)	• Enables processing of ultra-long contexts (128k+) that would overflow a single GPU's KV VRAM.	• Requires complex ring-attention communications to calculate global softmax scores.	Specialized long-context generation pipelines, stacked on top of TP/PP.

Interview Questions & Production Trade-offs

- What problem does this solve?
- Why was it introduced?
- What are its limitations?
- Computational Complexity (Time & Memory)
- Component Variable Denotation Legend (Explicitly defining $N, L, |V|, d, m, K, T, C, P$)
- Production Use Cases
- Follow-up questions interviewers ask

06. Speculative & Accelerated Decoding: Parallel Verification and Draft Models

Autoregressive token generation requires loading a model's entire weights from high-bandwidth memory (HBM) to GPU SRAM for every single token produced. Speculative decoding bypasses this memory bottleneck by generating multiple candidate tokens using a computationally cheap draft mechanism and verifying them in parallel during a single forward pass of the main target model.

---

1. Speculative Decoding Theory

Speculative decoding relies on two models:
1. **Draft Model (q)**: A small, fast model (e.g. Llama 1B) that generates tokens quickly.
2. **Target Model (p)**: A large, accurate model (e.g. Llama 70B) whose outputs we want to match.

At each speculative step:
1. The draft model runs autoregressively for γ steps to generate a draft sequence:

$$X_{\text{draft}} = (x_1, x_2, \ldots, x_\gamma)$$

2. The target model evaluates the entire block of draft tokens in a single parallel forward pass, computing the target probabilities for each position:

$$P_{\text{target}} = (p_1, p_2, \ldots, p_{\gamma+1})$$

3. The engine uses **rejection sampling** to verify each draft token sequentially.

---

2. Rejection Sampling: The Coin-Toss Intuition

Speculative decoding is **mathematically lossless**, meaning the generated tokens are identical to what the large target model would have output on its own. It guarantees this by utilizing a corrected rejection sampling protocol.

The Verification Protocol

For a candidate token x^* proposed by the draft model distribution $q(x)$:
1. **Acceptance Probability**: We accept x^* with a probability scaled by the density ratio:

$$\alpha = \min\left(1, \frac{p(x^*)}{q(x^*)}\right)$$

2. **Acceptance**: If accepted, we keep the token and check the next drafted token.
3. **Rejection**: If rejected, we throw away all subsequent drafted tokens, and sample a replacement token from the normalized difference distribution:

$$p'(x) = \frac{\max(0, p(x) - q(x))}{\sum_y \max(0, p(y) - q(y))}$$

Why This Works (The Intuition)

Think of verification as a biased coin-toss:
- **Case A: Target model likes the token *more* ($p(x^*) \geq q(x^*)$)**: The ratio is ≥ 1.0, so we accept it 100% of the time.
- **Case B: Target model likes the token *less* ($p(x^*) < q(x^*)$)**: The ratio is < 1.0, so we flip a coin with an acceptance probability equal to that ratio.
- **Fallback**: If the coin comes up tails (rejection), the target model selects a token from the difference distribution, which represents the tokens the target model preferred but the draft model under-sampled.

This dynamic correction guarantees that the final selection probability is exactly $p(x)$, aligning perfectly with the target model's original distribution while bypassing autoregressive compute latency.

---

3. Modern Speculative Architectures

Modern engines use advanced draft mechanisms to bypass tokenizer mismatches and dual-model VRAM overheads:

```
 <div style="display: flex; flex-direction: column; gap: 16px; font-family: 'Segoe UI',
sans-serif; margin: 20px 0;">
  <!-- EAGLE -->
  <div style="border: 1px solid #3b82f6; border-radius: 6px; padding: 12px; background-
color: #f8fafc;">
    <h4 style="margin: 0 0 6px 0; color: #1e3a8a; font-size: 14px;">1. EAGLE-2 / EAGLE-
```

```
3 (Feature Speculation)</h4>
    <div style="font-size: 12px; color: #475569; margin-bottom: 4px;">
      Instead of tokens, EAGLE drafts hidden feature vectors at the target model's
second-to-last layer. It processes candidates using a dynamic tree structures and
parallel tree attention.
    </div>
  </div>

  <!-- Medusa -->
  <div style="border: 1px solid #10b981; border-radius: 6px; padding: 12px; background-
color: #f8fafc;">
    <h4 style="margin: 0 0 6px 0; color: #065f46; font-size: 14px;">2. Medusa (Multi-
Head Prediction)</h4>
    <div style="font-size: 12px; color: #475569; margin-bottom: 4px;">
      Appends multiple linear prediction heads to the target model's output layer. Head
$k$ predicts the token $t+k$ concurrently, verifying candidates using a pre-computed
validation tree mask.
    </div>
  </div>

  <!-- Prompt Lookup -->
  <div style="border: 1px solid #f59e0b; border-radius: 6px; padding: 12px; background-
color: #f8fafc;">
    <h4 style="margin: 0 0 6px 0; color: #92400e; font-size: 14px;">3. Prompt-Lookup
Speculation (Zero-VRAM / Zero-Train)</h4>
    <div style="font-size: 12px; color: #475569; margin-bottom: 4px;">
      Scans prompt history for repeating N-grams. If the current sequence matches a
past N-gram sequence, the engine drafts the trailing tokens directly from context
history, requiring no helper draft models.
    </div>
  </div>
</div>
```

Speculative Drafting Strategy Comparison

Strategy	Drafting Mechanism	Pros	Cons	Production Choice
Draft Model	Small auxiliary autoregressive model (e.g. 1B) runs ahead.	• Robust; handles general conversational structures well.	• Requires loading a second model in VRAM. • Tokenizer alignment mismatches.	Standard baseline; ideal if helper model is small enough to fit in remaining memory.

Strategy	Drafting Mechanism	Pros	Cons	Production Choice
Medusa / EAGLE	Helper prediction heads or feature vectors inside the main model.	• Zero tokenizer issues. • Minimal extra VRAM (no second model loaded).	• Requires custom training and alignment tuning for the heads.	Custom high-performance serving environments (e.g., enterprise APIs).
Prompt-Lookup	Scans prompt/context history for repeating N-grams and copies them.	• **Zero VRAM & Zero Training**: Plug-and-play compatibility. • Minimal latency overhead.	• Limited to highly repetitive text styles (e.g. code generation, repetitive system logs).	Excellent for code completion servers and document-retrieval RAG bots.

---

4. Expected Tokens & Speedup Math

The effectiveness of speculative decoding is governed by the **Average Acceptance Rate** ($\alpha \in [0, 1]$), representing the probability that a draft token is accepted by the target model.

If the draft length is γ, the expected number of tokens generated per target forward pass is:

$$E[\text{tokens}] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

- If $\alpha = 0.8$ and $\gamma = 5$:

$$E[\text{tokens}] = \frac{1 - 0.8^6}{1 - 0.8} = \frac{1 - 0.262144}{0.2} = 3.689 \text{ tokens per forward pass}$$

- If $\alpha = 0.5$ and $\gamma = 5$:

$$E[\text{tokens}] = \frac{1 - 0.5^6}{1 - 0.5} = \frac{1 - 0.015625}{0.5} = 1.968 \text{ tokens per forward pass}$$

As α drops, the speedup factor decreases. If the draft model overhead exceeds the gains from multi-token parallel verification, speculative decoding can run slower than raw autoregressive decoding.

Interview Questions & Production Trade-offs

- What problem does this solve?
- Why was it introduced?
- What are its limitations?
- Computational Complexity (Time & Memory)
- Component Variable Denotation Legend (Explicitly defining $N, L, |V|, d, m, K, T, C, P$)
- Production Use Cases
- Follow-up questions interviewers ask

07. Inference Engines: Framework Architectures and Capabilities

Deploying LLMs in production requires selecting the appropriate inference serving framework. Serving engines manage memory allocation, request scheduling, batching execution, and hardware kernel mapping. Each stack targets specific hardware constraints and operational SLAs.

---

1. Deep-Dive Architecture Breakdown

vLLM

vLLM is an open-source, high-throughput LLM serving engine. It introduced **PagedAttention** to eliminate VRAM memory fragmentation.
- **Core Features**: Logical/physical block KV Cache manager, continuous batching scheduler, block hash prefix caching, ray-based tensor parallelism, and broad model coverage.
- **Optimal Environment**: Multi-GPU cluster serving (A100/H100/L4) for generic APIs with high concurrency.

SGLang (Structured Generation Language)

SGLang focuses on optimizing fast execution for complex prompting, structured generation (JSON/schema), and multi-agent workflows.
- **Core Features**: **RadixAttention** prefix caching (radix tree representation), **XGrammar** compiled structured decoding masks, and high-performance pipeline parallelism for Mixture-of-Experts (MoE) architectures (e.g. Mixtral/DeepSeek).
- **Optimal Environment**: Multi-turn agent loops, structured extraction pipelines, and models with massive system prompts.

TensorRT-LLM

NVIDIA's proprietary, high-performance compiler and engine stack for LLM serving.
- **Core Features**: Ahead-of-time (AOT) CUDA graph compilation, custom NVIDIA Tensor Core kernels, highly optimized FP8 execution, and in-flight batching.
- **Trade-Off**: Requires a separate compilation step to build engine binaries (`trtllm-build`), leading to high compilation times and strict hardware binding (the compiled engine only runs on the exact GPU model used for compilation).

- **Optimal Environment**: High-volume, enterprise single-model serving on NVIDIA Hopper (H100/H200) clusters where minimizing TPOT is the primary constraint.

llama.cpp & Ollama

A C++ implementation of transformer architectures optimized for local consumer hardware.
- **Core Features**: Block-wise quantization formats (GGUF), CPU/GPU heterogeneous memory-mapped execution, Metal API support (Apple Silicon), and zero external dependencies.
- **Optimal Environment**: Edge serving, developer local test setups, and CPU-only cloud instances.

Text Generation Inference (TGI)

Created by Hugging Face, TGI was one of the early production-grade engines. While it introduced key features like continuous batching and streaming tokens, it has transitioned to maintenance mode as open-source frameworks like vLLM and SGLang have overtaken its performance.

---

2. Comprehensive Engine Comparison Matrix

The table below compares serving engines across operational dimensions:

Feature Dimension	vLLM	SGLang	TensorRT-LLM	llama.cpp / Ollama
Primary Bottleneck	Memory Bandwidth	Scheduling Overheads	Network Bandwidth	Hardware Access Rate
Prefix Caching	Block-Level Hashing	Radix Tree (RadixAttention)	None (Static)	Local context caching
Structured Output	RegEx/JSON Outlines	Compiled XGrammar	Simple JSON Schema	Token-level constraints
Quantization Support	FP8, AWQ, GPTQ, INT8	FP8, AWQ, GPTQ, INT8	FP8, INT8/INT4 (SmoothQuant)	GGUF (K-Quant)
Compilation Overhead	Zero (JIT Python/C++)	Zero (JIT C++)	High (Pre-compile required)	Zero (Direct binary run)
Multi-GPU TP/PP	Native (Ray)	Native (Pytorch/Ray)	Custom MPI/NCCL	CPU/GPU load splits
Preferred Hardware	H100, A100, L4	H100, A100, L4	H100, H200	Apple Silicon, CPUs, Edge

---

Interview Questions & Production Trade-offs

- What problem does this solve?
- Why was it introduced?
- What are its limitations?
- Computational Complexity (Time & Memory)
- Component Variable Denotation Legend (Explicitly defining $N, L, |V|, d, m, K, T, C, P$)
- Production Use Cases
- Follow-up questions interviewers ask

08. Production Deployment: Architecture, Routing, and Scalability

Deploying LLMs at scale requires wraping inference engines within enterprise-grade deployment topologies. Serving platforms must manage rate limits, coordinate failover strategies, partition hardware resources across multiple tenants, and roll out model updates without causing system downtime or violating latency SLAs.

---

1. Enterprise Serving Topologies

Production workloads are split into two execution topologies:

Online Real-Time Serving

- **Workload**: Dynamic, user-facing applications (chatbots, real-time typing assistance) requiring low latencies.
- **Protocol**: Server-Sent Events (SSE) or WebSockets to stream generated tokens to the client as they are produced, minimizing perceived latency (TTFT).
- **Target SLA**: Lower TTFT (< 200 ms) and comfortable TPOT (< 30 ms).

Offline Batch Processing

- **Workload**: High-volume, non-interactive tasks (document classification, database summarization, embedding generation).
- **Protocol**: REST batch APIs or background queues (e.g. Celery). Schedulers combine inputs into massive batches to maximize GPU memory throughput.
- **Target SLA**: Tokens per Second (TPS) throughput, ignoring individual request latency.

---

2. Gateway Router & Provider Fallbacks

To ensure high availability, enterprise gateways abstract self-hosted engines (vLLM/SGLang clusters) and commercial APIs behind a unified API layer:

```
 <div style="font-family: 'Segoe UI', sans-serif; border: 1.5px solid #64748b; border-radius: 8px; background-color: #f8fafc; padding: 16px; margin: 20px 0;">
  <!-- Gateway Router Header -->
  <div style="background-color: #1e293b; color: white; padding: 8px; border-radius: 4px; font-weight: bold; font-size: 13px; text-transform: uppercase; letter-spacing: 0.05em; text-align: center; margin-bottom: 16px;">
    Unified Gateway Router (API Gateway)
  </div>

  <div style="display: flex; justify-content: space-between; align-items: stretch; gap: 12px; font-size: 11px;">
    <!-- Local Stack -->
    <div style="flex: 1; border: 1px solid #2563eb; background-color: #eff6ff; padding: 10px; border-radius: 6px; text-align: center;">
      <div style="font-weight: 700; color: #1e40af; margin-bottom: 6px; text-transform: uppercase;">Primary Route (Self-Hosted)</div>
      <div style="color: #1e3a8a; margin-bottom: 4px; font-weight: 600;">vLLM / SGLang Cluster</div>
      <div style="color: #64748b;">Lowest unit cost, secure data boundary</div>
    </div>

    <!-- Switch Logic -->
    <div style="display: flex; flex-direction: column; justify-content: center; align-items: center; width: 100px; text-align: center;">
      <span style="font-size: 9px; font-weight: bold; color: #e11d48; margin-bottom: 4px;">If 429 / Timeout</span>
      <div style="width: 100%; height: 3px; background-color: #cbd5e1; position: relative;">
        <div style="position: absolute; right: 0; top: -3px; border-top: 4px solid transparent; border-bottom: 4px solid transparent; border-left: 6px solid #64748b;">
</div>
      </div>
    </div>

    <!-- Fallback Stack -->
    <div style="flex: 1; border: 1px solid #10b981; background-color: #ecfdf5; padding: 10px; border-radius: 6px; text-align: center;">
      <div style="font-weight: 700; color: #065f46; margin-bottom: 6px; text-transform: uppercase;">Fallback Route (Commercial)</div>
      <div style="color: #047857; margin-bottom: 4px; font-weight: 600;">OpenAI / Gemini / Anthropic</div>
      <div style="color: #64748b;">High reliability, pay-per-token limits</div>
    </div>
  </div>
</div>
```

- **Failover Logic**: If the self-hosted engine throws a 503 (overloaded) or 429 (rate limit), the router catches the exception and redirects the query to a fallback commercial endpoint.
- **Model Redirection**: Queries can be routed dynamically based on prompt classification: complex reasoning prompts go to expensive target models, while simple classification queries go to smaller, quantized self-hosted models.

---

3. Multi-Tenant Resource Isolation

Sharing a single GPU cluster across multiple business units requires enforcing resource boundaries to prevent one tenant's queries from degrading the performance of others:
- **Rate-Limiting Semaphores**: Limit the number of concurrent active prefill and decode slots a single tenant can occupy.
- **VRAM Cache Quota Allocations**: Restrict the maximum KV Cache block allocations a single tenant's requests can hold in the active block table.
- **Priority Queues**: Place requests into prioritized tiers. Under heavy load, the scheduler evicts or preempts low-priority decoding iterations to allocate block capacity to high-priority requests.

---

4. Blue/Green and Canary Deployments

Model updates require swapping weights on active GPU memories without dropping connections.

Blue/Green Deployments

Two identical hardware environments are maintained. The **Blue** cluster runs the active production model (e.g. Model V1). The **Green** cluster is updated with the new model (e.g. Model V2).
Once Green passes verification, the load balancer shifts traffic to the Green cluster, and Blue is decommissioned.
- **Cons**: Requires doubling the active GPU allocation during transition.

Canary Deployments

Instead of an instant swap, a small fraction of production traffic (e.g. 5%) is routed to the new model instance.
- **Shadow Traffic**: Production requests are duplicated at the gateway. The duplicate request is processed by the new model in the background, comparing its outputs, latency, and error rates against production without returning its responses to the user.
- **Regression Triggers**: The gateway monitors output metrics (perplexity drift, TTFT spikes, guardrail violations). If any metric violates safety bounds, traffic is automatically routed back to the production instance.

Interview Questions & Production Trade-offs

- What problem does this solve?
- Why was it introduced?
- What are its limitations?
- Computational Complexity (Time & Memory)
- Component Variable Denotation Legend (Explicitly defining $N, L, |V|, d, m, K, T, C, P$)
- Production Use Cases
- Follow-up questions interviewers ask

09. Performance Monitoring, Profiling & Debugging: Production Incidents

Running LLM serving clusters at scale requires continuously monitoring infrastructure telemetry and system metrics. Because LLM workloads dynamically transition between compute-bound and memory-bound phases, diagnosing production incidents—such as latency spikes, memory crashes, and poor hardware utilization—demands a structured debugging protocol.

---

1. Operational Telemetry Metrics

To maintain health and reliability, serving platforms monitor two categories of metrics:

Latency & Quality Metrics

- **TTFT (p50, p95, p99)**: High p99 TTFT values indicate queue saturation or prefill scheduling bottlenecks.
- **TPOT / ITL (p50, p95, p99)**: The inter-token generation speed. Steady TPOT values confirm stable memory reads.
- **End-to-End Latency**: The total time from user submission to final response delivery.
- **Token Generation Ratio**: The ratio of generated tokens to prompt tokens, which helps project workload evolution.

Hardware & Memory Metrics

- **GPU HBM Memory Utilization**: The fraction of High Bandwidth Memory occupied.
- **KV Cache Block Allocation / Hit Rate**: The percentage of physical blocks allocated to cached sequences. A high hit rate indicates efficient prefix sharing.
- **Model FLOPs Utilization (MFU)**: The ratio of attained execution FLOP/s to the GPU's peak theoretical FLOP/s.
- **CUDA Memory Fragmentation Ratio**: The ratio of requested memory allocations to the largest contiguous free memory block.

---

2. Debugging Real-World Production Incidents

Below is a structured diagnostic guide for the three most common production LLM serving issues:

Incident A: CUDA Out of Memory (OOM) Errors under Load

Root Causes

1. **KV Cache Over-allocation**: The scheduler allocates too many physical blocks relative to the maximum sequence length, leading to memory exhaustion under high concurrency.
2. **Activation Spikes**: Compute-heavy prefill operations for long prompts create massive intermediate activation matrices in SRAM/HBM.
3. **Memory Leaks**: Non-contiguous allocations fragment CUDA memory, preventing the engine from allocating space for new request blocks.

Diagnostic Checklist

1. Inspect the stack trace: Did the crash occur during the prefill phase or the decode phase?
2. Check `max_model_len` and `gpu_memory_utilization` parameters in your configuration.
3. Query `nvidia-smi` or PyTorch's `memory_allocated()` to trace active allocations.
4. Set the vLLM parameter `max_num_seqs` to restrict the concurrent batch size.

Incident B: TTFT Latency Spikes (e.g. p99 TTFT > 10 Seconds)

Root Causes

1. **Queue Saturation**: A surge in request volume saturates the prefill queue, leaving incoming requests waiting for GPU scheduling slots.
2. **Un-cached System Prompts**: Multi-turn chats or RAG prompts with large context prefixes bypass the prefix cache, requiring the engine to recompute the entire prompt prefill.
3. **Head-of-Line Blocking**: Long prefill requests block active decoding steps.

Diagnostic Checklist

1. Inspect the scheduler queue status: Measure the wait time in the queue vs. the active compute time.

2. Verify the prefix cache hit rate. If the hit rate is low, inspect why prompts are changing slightly (e.g., dynamic timestamps or user IDs in system messages).
3. Enable **Chunked Prefill** to split long prompts and interleave prefill compute with active decode iterations.

Incident C: Low Throughput & GPU Under-utilization (Low MFU)

Root Causes

1. **Small Batch Sizes**: Under low traffic volume, the engine cannot form large batches, forcing the GPU to execute memory-bandwidth-bound single-token decodes.
2. **CPU-GPU Transfer Overheads**: Repeatedly transferring data between host memory (RAM) and GPU VRAM stalls Tensor Core processing.
3. **Un-batched Tool Calls**: Synchronously waiting for external API queries halts the generation loop.

Diagnostic Checklist

1. Check the active batch size. If traffic is low, consider deploying smaller, quantized models to reduce serving costs.
2. Use **CUDA Graphs** to record and execute sequences of GPU kernels, bypassing CPU-launch overheads.
3. Ensure tool execution and routing are asynchronous.

---

3. Profiling Tools

To diagnose deep performance bottlenecks, developers use profiling frameworks:
- **PyTorch Profiler**: Captures CPU/GPU execution trace files, identifying slow operators and memory allocation spikes.
- **NVIDIA Nsight Systems (`nsys`)**: A system-wide profiling tool that captures CUDA kernel execution timelines, NVLink communication overheads, and hardware stalls.
- **Prometheus vLLM endpoints**: vLLM exposes `/metrics` endpoints tracking current queue sizes, KV cache block hit rates, and iteration latencies, which can be visualized in Grafana dashboards.

Interview Questions & Production Trade-offs

- What problem does this solve?

- Why was it introduced?
- What are its limitations?
- Computational Complexity (Time & Memory)
- Component Variable Denotation Legend (Explicitly defining $N, L, |V|, d, m, K, T, C, P$)
- Production Use Cases
- Follow-up questions interviewers ask

🎯 LLM Inference & Optimization: High-Frequency Question Bank

> **Target Role:** AI Engineer / Applied AI Engineer / LLM Engineer (3+ Years Experience)
> **Scope:** Two-phase execution, KV Cache VRAM math, PagedAttention, quantization mechanics, continuous batching, speculative tree decoding, vLLM/SGLang engines, PD disaggregation, and production incident debugging.

---

1. Inference Fundamentals (8 Questions)

1. End-to-End Inference Pipeline

Explain the complete LLM inference pipeline step-by-step: Prompt $\rightarrow$ Tokenization $\rightarrow$ Embedding $\rightarrow$ Prefill GEMM $\rightarrow$ Autoregressive Decode GEMV $\rightarrow$ Sampling $\rightarrow$ Detokenization.
- **Short Answer**: The inference pipeline consists of processing the text input into discrete tokens, mapping them to embeddings, executing a parallel prefill phase (compute-bound matrix-matrix multiplication) to ingest the prompt and allocate the KV Cache, running sequential decode loops (memory-bandwidth-bound matrix-vector multiplication) to generate successive tokens, applying sampling logits transformations to select the next token, and detokenizing the resulting IDs back to human-readable characters.
- **Key Interview Points**: Prefill GEMM vs. Decode GEMV, KV Cache allocation, token-to-embedding mapping, and sampling constraints.
- **Technical Intuition**: Text is mapped to token IDs via BPE/SentencePiece. Embeddings convert these IDs to vector spaces $X \in \mathbb{R}^{B \times L \times d_{\text{model}}}$. Prefill calculates self-attention for all tokens in parallel using general matrix multiplication (GEMM) since the entire sequence is available. Decode computes queries, keys, and values for the new token, querying the cached KV tensors to compute single-token attention via matrix-vector products (GEMV).
- **Production Perspective**: Ensure detokenizers support streaming Server-Sent Events (SSE) so users receive tokens incrementally instead of waiting for the full output block.
- **Follow-up**: *What causes detokenization lag?* (Mainly buffering issues with multi-byte UTF-8 characters split across token boundaries).

2. Prefill Phase vs. Decode Phase

What are the fundamental computational and execution differences between the Prefill (Prompt Processing) phase and the Decode (Token Generation) phase?

- **Short Answer**: The Prefill phase processes all prompt tokens in parallel, saturating GPU Tensor Cores via compute-bound General Matrix Multiplications (GEMM). The Decode phase generates one token at a time sequentially, loading the entire model's weights and past KV caches for each step, which makes it memory-bandwidth-bound General Matrix-Vector Multiplication (GEMV).
- **Key Interview Points**: Compute-bound vs. memory-bandwidth-bound, GEMM vs. GEMV kernels, parallel input vs. sequential output.
- **Technical Intuition**: During prefill, the query, key, and value shapes are $[B, L_{\text{prompt}}, d_{\text{model}}]$. Since $L_{\text{prompt}} \gg 1$, matrix multiplications reuse model weights, yielding high arithmetic intensity. During decode, the query shape is $[B, 1, d_{\text{model}}]$, requiring the GPU to load the entire layer weights from HBM to SRAM to perform a single vector-matrix multiply.
- **Production Perspective**: Optimizing serving requires adjusting batching policies. Heavy prefill batches should be chunked to avoid halting active decode cycles.
- **Follow-up**: *Why does prefill have higher GPU power draw?* (Because Tensor Cores are active continuously, whereas decode leaves them mostly idle while waiting for memory transfers).

3. Arithmetic Intensity & Roofline Bottlenecks

Define Arithmetic Intensity **(FLOPs/Byte). Use the Roofline model to prove why the prefill phase is compute-bound (GEMM) while the decode phase is memory-bandwidth-bound (GEMV).**

- **Short Answer**: Arithmetic Intensity (I) is the ratio of FLOPs executed to memory bytes read/written. The Roofline Model states performance is bounded by $\min(P_{\text{peak}}, I \cdot B_{\text{peak}})$. For prefill, $I \approx L_{\text{prompt}}$, exceeding the hardware ridge point and making it compute-bound. For decode, $I \approx 1$ FLOP/Byte, which is well below the ridge point, making it memory-bandwidth-bound.
- **Key Interview Points**: Arithmetic Intensity formula, Roofline Model regions, Hardware Ridge Point calculation, GEMM vs. GEMV bounds.
- **Technical Intuition**: Given a model of size P parameters:
- Prefill (N tokens): $\text{FLOPs} = 2PN$, $\text{Bytes} = 2P$ (weights) + cache writes. Thus, $I \approx N$. For $N = 1024$ and FP16 weights, $I = 1024$ FLOPs/Byte.
- Decode (1 token): $\text{FLOPs} = 2P$, $\text{Bytes} = 2P$ (weights) + cache reads. Thus, $I \approx 1$.
Since an NVIDIA A100 has a ridge point of ≈ 156 FLOPs/Byte, prefill ($I = 1024 > 156$) is compute-bound, while decode ($I = 1 < 156$) is memory-bandwidth-bound.
- **Production Perspective**: Increasing memory bandwidth (e.g. upgrading H100 to H200 with faster HBM3e) directly improves decode throughput, while compute upgrades improve prefill speeds.
- **Follow-up**: *How does quantization affect arithmetic intensity?* (Quantizing to INT8/INT4 halves or

quarters the denominator, increasing I and moving decode execution closer to the compute-bound boundary).

4. Latency & Throughput Metrics

Define TTFT (Time To First Token), TPOT (Time Per Output Token / Inter-Token Latency), TPS (Tokens Per Second), and End-to-End Latency. When is each metric critical for user SLAs?
- **Short Answer**:
- **TTFT**: Delay until the first token is received. Critical for real-time user-facing chat.
- **TPOT (ITL)**: Average time between successive tokens. Critical for reading comfort.
- **TPS**: Total token generation throughput. Critical for background batch jobs.
- **End-to-End Latency**: Total time for request completion. Critical for non-streaming programmatic API integration.
- **Key Interview Points**: Latency component definitions, client experience mappings, and throughput metrics.
- **Technical Intuition**: $\text{TTFT} = \text{Queue Delay} + \text{Prefill Latency}$. $\text{TPOT} = \text{Decode Iteration Latency}$. $\text{End-to-End Latency} = \text{TTFT} + (L_{\text{output}} - 1) \times \text{TPOT}$.
- **Production Perspective**: Trade-off batch sizes: larger batches increase throughput (TPS) but degrade latency (TTFT and TPOT).
- **Follow-up**: *Why do chat interfaces care more about TTFT than End-to-End latency?* (Because a low TTFT creates a responsive feel, hiding the total generation time).

5. Prefill-Decode (PD) Disaggregation

What is Prefill-Decode Disaggregation, and why does splitting compute-bound prefill nodes from memory-bandwidth-bound decode nodes across NVLink/RDMA networks prevent head-of-line blocking?
- **Short Answer**: PD Disaggregation separates physical GPUs into dedicated prefill workers and decode workers. This prevents compute-heavy prompt processing steps from blocking active, iteration-level decode streams in the serving queue, resolving head-of-line blocking and stabilizing inter-token latency (TPOT).
- **Key Interview Points**: Head-of-line blocking, GPU workload isolation, KV Cache transfer cost, NVLink/RDMA fabrics.
- **Technical Intuition**: In collocated execution, a new prefill request takes over GPU compute, stalling decode iterations of active sessions for several hundred milliseconds. By splitting them, prefill nodes calculate keys/values and transfer them via high-speed RDMA networks directly to the decode nodes' memory pool, ensuring decode nodes execute uninterrupted.
- **Production Perspective**: PD disaggregation is highly beneficial for long-context workloads (e.g. RAG, document analysis) where prefill tasks are extremely heavy.

- **Follow-up**: *What is the main bottleneck in PD disaggregation?* (The network latency of transferring massive KV cache tensors between nodes).

6. Sequential Autoregressive Dependence

Why is autoregressive token generation inherently sequential, and why does this prevent standard intra-sequence parallelization during the decode phase?
- **Short Answer**: Autoregressive decoding relies on the causal dependency where each output token is conditioned on all preceding tokens: $P(x_t \mid x_{1:t-1})$. Because token x_t cannot be computed until the token ID of x_{t-1} is selected and appended to the input sequence, we cannot parallelize the execution of sequential tokens within a single request.
- **Key Interview Points**: Causal attention, sequence dependency, sequential decoding loop.
- **Technical Intuition**: Self-attention requires computing the query vector of the current step against the keys of all previous steps. Because the query vector q_t depends on the token embedding of x_{t-1}, the matrix operations for step t cannot begin until step $t-1$ completes.
- **Production Perspective**: To maximize hardware utilization despite sequence sequentiality, we use batching to process different sequences in parallel.
- **Follow-up**: *Can we generate multiple tokens at once losslessly?* (Only via Speculative Decoding, which validates multiple draft tokens in parallel).

7. Inference vs. Training Memory Footprints

Why is LLM inference generally more memory-bandwidth-bound and VRAM-sensitive per generated token than LLM pre-training or fine-tuning?
- **Short Answer**: During training, activation tensors are saved for backpropagation, and weights are reused across massive batches of tokens processed in parallel. During inference, we do not store gradients, but we must store the KV Cache for every concurrent request, which grows linearly with batch size and context length, making the VRAM footprint highly sensitive to dynamic traffic workloads.
- **Key Interview Points**: Activation storage, gradient caching, KV Cache footprint, weight reuse differences.
- **Technical Intuition**: Training uses $B \times L$ tokens per step, sharing weight overheads. Inference uses a batch size B but processes only 1 token per step, requiring the model to load all weights from HBM to SRAM for that single token's computation.
- **Production Perspective**: In training, we scale compute clusters to maximize FLOPs. In inference, we scale clusters primarily to accommodate the aggregate VRAM footprint of active KV Caches.
- **Follow-up**: *What is the activation memory footprint during prefill?* (It scales with the square of sequence length, which can trigger CUDA OOMs on extremely long prompts).

8. Latency Breakdown Factors

What hardware and algorithmic factors contribute most to TTFT spikes vs. TPOT slowdowns in production LLM clusters?

- **Short Answer**: TTFT spikes are caused by queue wait times, long prompt lengths (prefill computation), and uncached system prompts. TPOT slowdowns are caused by HBM bandwidth limits, high active batch sizes (requiring large weight/KV cache memory reads), and inter-GPU communication latency in Tensor Parallel setups.
- **Key Interview Points**: Queue delays, HBM bandwidth constraints, tensor parallel comms (All-Reduce overheads), and prefix cache misses.
- **Technical Intuition**: $\text{TTFT} \propto \frac{\text{Prompt Length} \times \text{Layers}}{\text{Compute FLOP/s}} + \text{Queue Delay}$. $\text{TPOT} \propto \frac{\text{Model Weights} + \text{KV Cache Size}}{\text{HBM Bandwidth}} + \text{Communication Latency}$.
- **Production Perspective**: To stabilize TTFT, deploy prefix caching and chunked prefill. To stabilize TPOT, use GQA, quantization (FP8/INT4), and optimize TP communications.
- **Follow-up**: *How does TP width affect TPOT?* (Increasing TP width reduces compute time per GPU but increases All-Reduce communication overhead. Beyond a certain width, communication latency outweighs compute gains, slowing down TPOT).

---

2. Decoding & Sampling (8 Questions)

9. Deterministic vs. Stochastic Sampling

Compare Greedy Decoding, Beam Search, Top-K, Top-p (Nucleus), and Min-p **sampling across output quality, diversity, and computational overhead.**

- **Short Answer**: Greedy decoding and Beam Search are deterministic; they produce high-factuality, low-diversity outputs. Beam Search has high memory overhead due to maintaining B active KV caches. Top-K, Top-p, and Min-p are stochastic; they introduce creativity. Top-K uses a static cutoff count, Top-p dynamically adjusts the cutoff based on cumulative probability, and Min-p dynamically filters based on probability relative to the top token.
- **Key Interview Points**: Deterministic vs. stochastic, beam search memory footprint, static vs. dynamic truncation thresholds.
- **Technical Intuition**:
- Greedy: $x_t = \arg\max P(x)$.
- Beam Search: Tracks B paths, maximizing $\sum \log P(x)$.
- Top-K: Sorts vocabulary, samples from top K candidates.
- Top-p: Samples from smallest set V where $\sum_{i \in V} P_i \geq p$.
- Min-p: Samples from tokens where $P_i \geq P_{\max} \times p_{\text{min\_threshold}}$.
- **Production Perspective**: Min-p is preferred for production chatbots because it maintains quality under high temperatures while preserving creative vocabulary options.

- **Follow-up**: *Why is Beam Search avoided in chat servers?* (Because it requires duplicating the KV cache B times for each concurrent request, exhausting VRAM).

10. Temperature Scaling Mechanics

Mathematically explain how Temperature (T) modifies the raw logit vector before the Softmax transformation ($P_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$). What happens as $T \to 0$ and $T \to \infty$?
- **Short Answer**: Temperature acts as a scaling divisor for the raw logits z_i. When $T \to 0$, the logit differences are amplified, making the distribution highly peaky and converging to deterministic greedy selection. When $T \to \infty$, the logit values converge to 0, making the exponentiated terms approach 1, which results in a uniform random distribution.
- **Key Interview Points**: Logit scaling, Softmax exponentiation, entropy changes.
- **Technical Intuition**: Given logits $z = [2.0, 1.0, 0.0]$:
- At $T = 1.0$: $P \approx [0.665, 0.245, 0.090]$.
- At $T = 0.5$ (low temp): scaled $z/T = [4.0, 2.0, 0.0] \implies P \approx [0.867, 0.117, 0.016]$ (higher confidence).
- At $T = 2.0$ (high temp): scaled $z/T = [1.0, 0.5, 0.0] \implies P \approx [0.506, 0.307, 0.186]$ (flatter).
- **Production Perspective**: Set $T = 0.0$ (or greedy) for code generation, JSON parsing, and mathematical calculations. Use $T = 0.7 - 0.9$ for creative text generation.
- **Follow-up**: *Does temperature change the ranking of tokens?* (No, because the division by $T > 0$ is a monotonic transformation. It only alters their relative probabilities).

11. Top-K vs. Top-p vs. Min-p Truncation

Contrast static Top-K, cumulative Top-p, and dynamic confidence-relative Min-p **truncation ($p_{\text{threshold}} = p_{\max} \times \text{scale}$). Why is Min-p superior at preserving output coherence under high temperatures?**
- **Short Answer**: Top-K keeps a fixed number of tokens, ignoring the shape of the distribution. Top-p keeps a dynamic number of tokens based on cumulative probability, which can include low-probability tail tokens if the top token is weak. Min-p filters out tokens whose probability falls below a threshold relative to the top token's probability. This prevents low-confidence tail tokens from being sampled when the top token is highly confident, maintaining coherence under high temperatures.
- **Key Interview Points**: Fixed cutoffs vs. dynamic distributions, tail token suppression, dynamic confidence scaling.
- **Technical Intuition**: In a highly confident setting where the top token has $p_{\max} = 0.90$:
- Min-p (with scale 0.05) sets the threshold to 0.045. Any token with probability $< 4.5\%$ is pruned.
- Top-p (with threshold 0.95) might still include tokens down to 0.1% to fill the remaining 5%

cumulative probability.
This makes Min-p more effective at pruning low-quality tail tokens.
- **Production Perspective**: Deploying Min-p allows using higher temperatures (e.g. $T = 1.2$) for creative variety without suffering from grammar collapse or gibberish.
- **Follow-up**: *Can Top-K and Top-p be combined?* (Yes, traditionally models apply Top-K first, then Top-p, then Temperature. Min-p replaces Top-p).

12. Application-Specific Sampling Parameters

How would you configure temperature, top-p/min-p, and penalties for: Factual QA / Code Generation, Conversational Chatbots, Creative Writing & Brainstorming, and Structured JSON Data Extraction?
- **Short Answer**:
- **Factual QA / Code**: $T = 0.0$ (Greedy), no truncation, no penalties.
- **Structured JSON**: $T = 0.0$ with grammar-guided constraints (Outlines).
- **Conversational Chat**: $T = 0.7$, Top-p = 0.90 (or Min-p = 0.05), Repetition Penalty = 1.05.
- **Creative Writing**: $T = 1.1$, Min-p = 0.10, Presence Penalty = 0.10.
- **Key Interview Points**: Logit tuning, deterministic configurations, repetition penalty tuning.
- **Technical Intuition**: High factuality requires minimizing entropy (low temperature). Creative tasks require expanding the search space (high temperature, lower truncation thresholds). Repetition penalties prevent looping behaviors in open-ended generations.
- **Production Perspective**: Exposing these parameters via APIs allows clients to customize inference behavior per task domain.
- **Follow-up**: *Why do repetition penalties occasionally break JSON generation?* (Because they penalize repetitive structural characters like brackets and quotes, causing schema validation errors).

13. Logit Penalties

Explain the mathematical mechanics of Frequency Penalty, Presence Penalty, and Repetition Penalty during logit post-processing.
- **Short Answer**: Frequency and Presence penalties reduce logits linearly based on token occurrences. Frequency penalty scales with the count of appearances, while Presence penalty applies a constant deduction if the token appears at least once. Repetition penalty divides the logit by a scaling factor if the logit is positive, or multiplies it if negative, reducing the probability of repeating tokens.
- **Key Interview Points**: Linear vs. multiplicative penalties, frequency-dependent scaling, token presence tracking.
- **Technical Intuition**:
- Frequency / Presence Penalty:

$$z_i = z_i - (\text{count}_i \times \alpha_{\text{frequency}} + \delta(\text{count}_i > 0) \times \alpha_{\text{presence}})$$

- Repetition Penalty ($\theta \geq 1.0$):

$$z_i = \begin{cases} z_i / \theta & \text{if } z_i \geq 0 \\ z_i \cdot \theta & \text{if } z_i < 0 \end{cases}$$

- **Production Perspective**: Keep penalties subtle (e.g. $1.02 - 1.08$). High values can distort vocabulary distributions, causing grammar collapse.
- **Follow-up**: *Do these penalties apply to prompt tokens?* (Typically they only apply to generated output tokens, but some engines support penalizing prompt tokens as well).

14. Stop Sequences & Token Termination

How are stop sequences monitored during streaming generation, and how do engines handle multi-token stop sequences across token boundaries?
- **Short Answer**: Serving engines maintain a sliding buffer of recent output tokens. At each step, they check if the buffer suffix matches any configured stop sequence. If a stop sequence spans across multiple tokens, the engine buffers the potential match and delays sending the streaming response to the client until it confirmed a complete match or a mismatch.
- **Key Interview Points**: Sliding buffer validation, partial match buffering, streaming token delays.
- **Technical Intuition**: If the stop sequence is `"\nUser:"` and the model generates `"\n"`, then `"User"`, the engine buffers these tokens. If the next token is `":"`, the generation terminates and the buffered tokens are discarded. If the next token is `"agent"`, the engine flushes the buffered tokens (`"\nUser agent"`) to the stream.
- **Production Perspective**: Instruct client applications to handle partial stop sequence strings to avoid flickering in the user interface.
- **Follow-up**: *What happens if the model generates a stop sequence token but the engine misses it?* (The model continues generating text, causing prompt leaks or multi-turn sequence failures).

15. Constrained & Structured Decoding Overhead

How do context-free grammars (CFGs) and regex state-masking (e.g., using Outlines or XGrammar) enforce strict JSON schemas during decoding without causing massive per-token latency penalties?
- **Short Answer**: Constrained decoding compiles the target JSON schema or regex into a Finite State Machine (FSM). At each step, the FSM determines the valid next tokens, and the engine masks out invalid logits. To avoid per-token latency penalties, frameworks like XGrammar pre-compute token transition maps and cache them, keeping structured generation near native speeds.
- **Key Interview Points**: FSM token state transitions, logit masking ($P_i = -\infty$), transition map caching.

- **Technical Intuition**: The vocabulary is indexed by trie structures. When the FSM state requires a digit, the engine sets the logits of all non-digit tokens to $-\infty$. Doing this on the GPU via cached index lists keeps validation overhead under a few microseconds.
- **Production Perspective**: Structured generation guarantees that outputs are parseable, eliminating client-side retries and parsing crashes.
- **Follow-up**: *Can we enforce CFGs on closed APIs?* (Only via prompt engineering or schema parameters if supported. Logit masking requires direct access to the model's logits before sampling).

16. Beam Search Memory & Compute Footprint

Why is Beam Search rarely used for online conversational LLM serving despite its utility in machine translation?
- **Short Answer**: Beam Search maintains B candidate sequences (beams) at each step. This requires keeping B distinct KV Caches for a single request, multiplying memory usage by B. This memory footprint limits serving capacity and causes CUDA OOMs under high concurrency.
- **Key Interview Points**: KV Cache duplication, beam expansion complexity, memory overhead.
- **Technical Intuition**: For beam width $B = 4$, the engine must store 4 key-value states for every token generated. During beam expansion, if paths split, the engine must copy and reorganize the KV cache blocks in VRAM, which creates high memory-copy overhead and slows down TPOT.
- **Production Perspective**: Use Beam Search only for offline batch translation or document generation where quality is critical and latency is not a bottleneck.
- **Follow-up**: *Does Beam Search improve conversational quality?* (No, it often leads to repetitive, generic responses in open-ended conversations compared to stochastic sampling).

---

8. Production Deployment, Monitoring & Debugging (8 Questions)

48. High-Availability Serving Architecture

Design a production multi-region LLM serving infrastructure featuring dynamic model routing, semantic caching (GPTCache), blue-green model swaps, and zero-downtime failover.
- **Short Answer**: A high-availability LLM infrastructure uses a global CDN load balancer routing to regional gateways. Regional gateways check incoming queries against a semantic cache (e.g., Redis + GPTCache) to instantly serve cached responses for redundant queries. For cache misses, requests are routed to local vLLM/SGLang engine nodes. If regional queues saturate or error rates spike, gateways automatically failover queries to standby regions or fallback third-party APIs. Model updates are managed via blue-green swaps where traffic is routed to the new cluster only after successful health checks.
- **Key Interview Points**: Global vs. regional gateways, semantic caching matching, multi-region

failover latency, blue-green resource allocation.
- **Technical Intuition**: Semantic caches compute prompt embeddings and query a vector database: if cosine similarity ≥ 0.95, the cached completion is returned. Model routers track regional GPU active cache block usage and queue backlogs. Zero-downtime failover redirects traffic to fallback zones when regional latencies cross SLA boundaries, protecting the user experience.
- **Production Perspective**: Keep connection timeouts short (e.g., <500ms) on regional endpoints to ensure instant failover triggers under load spikes.
- **Follow-up**: *How do you synchronize prefix caches across nodes?* (You don't sync them dynamically; instead, route requests with identical prompt prefixes to the same GPU worker nodes to maximize local cache hits).

49. Provider Abstraction & Fallback Policies

How do you build a resilient provider abstraction layer that dynamically routes queries between self-hosted models (vLLM/SGLang) and cloud APIs (OpenAI/Anthropic/Gemini) based on latency, cost, and rate limits?
- **Short Answer**: Design a central gateway router middleware that wraps self-hosted and cloud LLM APIs behind a single unified interface. The router uses a priority-based routing policy: it routes queries to self-hosted vLLM clusters first to minimize token costs. If the self-hosted cluster returns 429 (rate limit), 503 (service unavailable), or if active latency monitoring detects TPOT crossing SLA limits, the router fails over to commercial cloud APIs dynamically.
- **Key Interview Points**: Resiliency middleware, unified schema parsing, rate-limit headers, cost-latency thresholding.
- **Technical Intuition**: The gateway tracks downstream worker health. It parses rate-limit headers (`x-ratelimit-remaining`) to preemptively avoid sending traffic to exhausted nodes. Fallback transitions are implemented using circuit breakers that pause routes to failing self-hosted clusters for a cooldown window.
- **Production Perspective**: Map self-hosted and third-party outputs to a unified Pydantic schema (e.g., using Instructor) to prevent parser failures during fallback events.
- **Follow-up**: *How do you minimize fallback latency?* (Parallelize requests by starting a standby fallback call if the primary self-hosted call does not return a token within a tight timeout window).

50. Blue-Green & Canary Deployments

How do you execute canary deployments for fine-tuned LLM service updates, and what automated regression checks (perplexity drift, output latency, guardrail pass rate) trigger rollback?
- **Short Answer**: Deploy the new model (Green) alongside the production model (Blue) on a small subset of serving nodes (e.g., 5% traffic). Use shadow traffic to duplicate production prompts to the Green instances in the background. Automated monitors calculate Green's metrics: perplexity drift

against calibration sets, TTFT/TPOT latency compliance, and guardrail pass rates. If any metric violates safety bounds, the gateway automatically rolls back traffic to the Blue instances.
- **Key Interview Points**: Shadow testing setups, metric evaluation pipelines, automatic rollback triggers.
- **Technical Intuition**: The gateway duplicates incoming payloads, executing inference on both models. Green's responses are analyzed but discarded, allowing the platform to verify performance under real-world traffic profiles without risking regression exposure to active users.
- **Production Perspective**: Canary testing is critical for LLMs because offline benchmark evaluations (like MMLU) often fail to capture subtle language drift or formatting regressions.
- **Follow-up**: *How do you measure perplexity dynamically?* (Perplexity requires calculating cross-entropy loss over logit values, which is only supported on self-hosted engines; for closed APIs, use LLM-as-a-judge consistency scoring).

51. Production Health & Telemetry Metrics

What specific operational metrics (p50/p95/p99 TTFT, TPOT/ITL, GPU HBM utilization, KV cache block hit rate, CUDA memory fragmentation) do you monitor continuously?
- **Short Answer**: We monitor p50/p95/p99 TTFT to evaluate prompt processing speeds and queue delays, TPOT/ITL to track output generation speeds, GPU HBM memory utilization to monitor overall capacity, KV Cache block hit rates to verify prefix caching reuse, and CUDA memory fragmentation ratios to identify potential memory exhaustion.
- **Key Interview Points**: Telemetry monitoring categories, latency distributions, memory usage indicators.
- **Technical Intuition**:
- High TTFT indicates prefill queue congestion or cache misses.
- Slow TPOT indicates HBM bandwidth bottlenecks or network overheads in Tensor Parallel clusters.
- Dropping KV Cache hit rates indicates prompt template drift or cache thrashing.
- CUDA fragmentation indicates non-contiguous memory allocations.
- **Production Perspective**: Configure Prometheus to scrape metrics directly from the serving engine (e.g., `/metrics` on vLLM) and compile them into Grafana dashboards.
- **Follow-up**: *What alert thresholds would you set for KV cache utilization?* (Trigger alerts when active block utilization exceeds 90% for sustained periods, indicating cache saturation and imminent preemption).

52. Debugging High TTFT Spikes

Your monitoring system flags p99 TTFT spikes reaching 10 seconds while TPOT remains normal. Walk through your systematic diagnostic process (prefill queue backlogs, un-cached prompt prefixes, un-chunked prefills).
- **Short Answer**: Identify the bottleneck by isolating the queue delay from the compute time:

1. Check the queue size: if the queue is large, the spike is caused by prefill task backlogs.
2. Check prefix cache hit rates: a sudden drop indicates that prompts are not matching cached prefixes, forcing the model to re-run full prefill compute passes.
3. If the queue is small but prefill time is high, the cause is un-chunked prefills of massive prompts. Enable chunked prefill to interleave prompt processing and stabilize TTFT.
- **Key Interview Points**: Queue delay vs. compute delay, cache hit rate metrics, head-of-line blocking resolution.
- **Technical Intuition**: Because prefill is compute-bound, processing a 32k prompt can block the GPU for several seconds. If multiple long prompts arrive simultaneously, they block all decode slots, driving up TTFT.
- **Production Perspective**: Prevent prefill bottlenecks by enforcing prompt length limits at the gateway and standardizing system prompt formats to maximize Radix tree matches.
- **Follow-up**: *How do you identify prompt prefix mismatch causes?* (Inspect prompt templates: check if dynamic values like timestamps, user IDs, or random seeds are being injected into the system prefix).

53. Debugging GPU Out-Of-Memory (OOM) Errors

A production LLM worker crashes with a CUDA OOM error under load. How do you investigate whether the root cause was activation spikes, KV cache over-allocation, or CUDA memory fragmentation?
- **Short Answer**: Analyze the crash trace and allocator state:
1. If the crash occurs during prefill, the cause is an activation memory spike from an excessively long prompt.
2. If it occurs during decode under high concurrency, the cause is KV cache over-allocation.
3. If the allocator logs show free VRAM bytes but no contiguous block of the requested size can be allocated, the cause is CUDA memory fragmentation.
- **Key Interview Points**: OOM timing checks, configuration bounds, allocation fragmentation patterns.
- **Technical Intuition**:
- Activation memory scales with sequence length and batch size ($O(L^2)$).
- KV Cache VRAM is bound by the engine's `gpu_memory_utilization` configuration.
- Fragmentation occurs when allocating and deallocating memory of varying sizes without contiguous mapping.
- **Production Perspective**: Set `gpu_memory_utilization` to 0.90 (leaving a 10% buffer for activations) and configure PyTorch's allocator split limits to reduce fragmentation.
- **Follow-up**: *How does PagedAttention prevent fragmentation?* (By virtualizing the KV cache memory space, allowing non-contiguous physical block allocations to act as a single logical sequence).

54. Diagnosing Poor GPU Utilization (Low MFU)

A GPU node shows 95% power consumption but Model FLOPs Utilization (MFU) is below 15%. What are the primary causes (small batch size, memory bandwidth saturation, CPU-GPU data transfer overhead)?

- **Short Answer**: The primary cause is memory bandwidth saturation during the decode phase. Under low batch sizes, the GPU Tensor Cores sit idle while the system spends cycles loading model weights from HBM to SRAM for single tokens. Other causes include CPU-GPU synchronization stalls and un-batched tool execution delays.
- **Key Interview Points**: Memory-bandwidth bound idle states, low batch size compute limits, CPU-GPU data transfer bottlenecks.
- **Technical Intuition**: The GPU draws peak power because the memory buses are fully saturated reading parameters from HBM. However, because $I \approx 1$ FLOP/Byte during decode, the Tensor Cores execute very few operations, keeping MFU extremely low.
- **Production Perspective**: To raise MFU, increase batch sizes using continuous batching or implement speculative decoding to execute more FLOPs per memory read.
- **Follow-up**: *What is a typical MFU during prefill vs. decode?* (Prefill MFU can reach 40-50% due to compute parallelism, while decode MFU is often below 5% for small batches).

55. Cost Optimization Framework

Walk through your exact strategy to reduce an enterprise LLM platform's monthly GPU cloud bill by 50% without dropping response quality or violating latency SLAs.

- **Short Answer**: Implement a four-stage optimization pipeline:
1. **Quantization**: Quantize model weights to FP8 or 4-bit AWQ, reducing model VRAM size and memory bandwidth pressure by 2x to 4x.
2. **Prefix Caching**: Enable RadixAttention prefix caching to bypass prefill compute on repeated system prompts and RAG contexts.
3. **Continuous Batching**: Use iteration-level scheduling to maximize throughput per GPU node.
4. **Auto-scaling**: Set up horizontal auto-scaling to scale down GPU nodes during low-traffic hours.
- **Key Interview Points**: Quantization cost benefits, cache efficiency gains, batch serving density, dynamic scaling.
- **Technical Intuition**: Moving from FP16 to INT4 AWQ allows running large models on cheaper, memory-bound GPUs (e.g. A10G instead of A100) while maintaining accuracy, directly cutting cloud spend.
- **Production Perspective**: Pair prefix caching with continuous batching to maximize the capacity of existing nodes before scaling cluster sizes.
- **Follow-up**: *How does semantic caching fit in this cost framework?* (Reclaims 10% to 30% of query costs by returning cached text from a vector database for matching inputs, bypassing LLM compute entirely).

---

9. System Design & Scenario Questions (5 Questions)

56. Design ChatGPT-Scale Inference Service

Design an inference platform capable of serving millions of concurrent active users with streaming SSE responses, prompt prefix caching, multi-tenant rate limits, and sub-second TTFT SLAs.
- **Short Answer**: Design a multi-tier serving architecture:
1. **Gateway Tier**: Handles rate-limiting (token bucket), routes users to regional endpoints, and executes semantic caching.
2. **Orchestrator Tier**: Inspects prompt headers and routes requests to model workers holding matching RadixAttention prefixes.
3. **Worker Tier**: Run vLLM or SGLang clusters configured with PagedAttention, continuous batching, and chunked prefills to maintain sub-second TTFT SLAs under heavy traffic.
- **Key Interview Points**: API gateway rate limiting, prefix-routing load balancing, continuous batching, and chunked prefill coordination.
- **Technical Intuition**: Directing prompts with identical system prefixes to the same worker node maximizes Radix tree cache matches. This reduces the prefill phase compute to a memory lookup, keeping p99 TTFT below target thresholds.
- **Production Perspective**: Expose telemetry metrics (Prometheus) on engines to monitor KV Cache usage and trigger horizontal auto-scaling before memory saturation.
- **Follow-up**: *How do you handle regional outages?* (The global load balancer automatically redirects traffic to healthy regions, while gateways apply fallback routing to alternative APIs).

57. Design Multi-Model Router Platform

Design an enterprise AI gateway supporting 20+ fine-tuned models with varying latency SLAs (sub-100ms vs. batch) and cost constraints.
- **Short Answer**: Build a gateway with a dynamic router and a shared model worker cluster:
- **Dynamic Router**: Routes incoming prompts to regional nodes based on SLAs and costs.
- **Shared Cluster**: Uses multi-LoRA serving (e.g., vLLM multi-LoRA) to serve 20+ fine-tuned models on a single GPU cluster. The base model is loaded in VRAM once, and custom LoRA adapters are dynamically loaded into memory on-demand.
- **Key Interview Points**: Multi-LoRA serving architectures, dynamic gateway routing parameters, cost-latency trade-offs.
- **Technical Intuition**: Standard serving requires loading one base model per GPU, which is memory-prohibitive for 20+ models. Multi-LoRA serving processes adapters as small parameter modifications, allowing the GPU to run multiple fine-tuned models concurrently.
- **Production Perspective**: Set routing thresholds: send low-latency queries to fine-tuned edge models, and high-complexity queries to base models with LoRA adapters.

- **Follow-up**: *What is the latency overhead of LoRA adapter swapping?* (Loading adapters from host RAM to GPU memory can introduce minor latency, which is mitigated by caching active adapters in VRAM).

58. GPU Memory Bottleneck Priority Checklist

You have a single 24GB GPU and need to serve an 8B model at 32k context length. Walk through your prioritized sequence of optimizations (GQA, INT4/FP8 quantization, PagedAttention, sliding window) to fit the workload.
- **Short Answer**: Follow this sequence:
1. **Weight Quantization**: Quantize weights to 4-bit AWQ. This reduces weight footprint from 16GB to 4GB, freeing up 12GB.
2. **PagedAttention**: Reclaim fragmented memory blocks, maximizing active cache space.
3. **Grouped-Query Attention (GQA)**: Ensure GQA is enabled (standard on Llama-3 8B), reducing KV Cache VRAM size by 8x.
4. **Sliding Window / Context Trimming**: Apply context window limits if sequence length exceeds VRAM capacity under load.
- **Key Interview Points**: Quantization savings, PagedAttention memory mapping, GQA reduction ratios, context limits.
- **Technical Intuition**:
- Weights: FP16 = 16GB, INT4 = 4GB (saving 12GB).
- KV Cache at 32k context ($b = 1$, Llama-3 8B GQA):

$$\text{VRAM}_{\text{KV}} = 2 \times 1 \times 32768 \times 32 \times 8 \times 128 \times 2 \approx 4 \text{ GB}$$

- The model (4GB) and KV cache (4GB) easily fit within the 24GB VRAM, leaving a 16GB buffer for activation spikes and batch concurrency.
- **Production Perspective**: Prioritize optimizations that maintain output quality (quantization, PagedAttention) before applying destructive window limits.
- **Follow-up**: *Can we run this setup without quantization?* (No, because FP16 weights (16GB) and 32k context cache (4GB) would consume 20GB, leaving insufficient memory for activations and batch concurrency).

59. Optimizing a 70B Latency SLA Violation

A 70B model fails its p95 TPOT SLA (requires 50 tokens/sec, achieves 20 tokens/sec). How do you optimize the system (tensor parallelism, AWQ/FP8 quantization, speculative decoding, continuous batching) before downgrading to an 8B model?
- **Short Answer**: Apply these optimizations systematically:
1. **Quantization**: Quantize the model to FP8 or INT4 (AWQ). This reduces weight data transfer sizes

by 2x-4x, accelerating memory reads.
2. **Tensor Parallelism (TP)**: Scale the TP width (e.g. from TP=2 to TP=4 or TP=8) across NVLink nodes to distribute compute and memory bandwidth loads.
3. **Speculative Decoding**: Implement a fast draft model (e.g. Llama-3 8B) or Prompt-Lookup to validate multiple candidate tokens in parallel, bypassing decode bandwidth limits.
4. **Continuous Batching**: Maximize batch utilization to raise average throughput.
- **Key Interview Points**: Quantization speedups, TP width scaling, speculative decoding acceleration.
- **Technical Intuition**:
- Quantizing to INT4 AWQ cuts model read traffic from 140GB/token to 35GB/token, directly increasing TPOT throughput.
- Scaling TP width to 4 nodes reduces weights loaded per node, raising memory-read speeds.
- Speculative decoding generates multiple tokens per target step, bypassing the sequential generation limit.
- **Production Perspective**: Evaluate the cost-latency tradeoff of scaling TP nodes before deciding to downgrade to a smaller model.
- **Follow-up**: *Does TP scaling always decrease TPOT?* (No, if the network interconnect is slow, the All-Reduce communication overhead can exceed compute savings, slowing down TPOT).

60. Fair Inference Benchmarking Methodology

How do you design a fair, reproducible benchmarking methodology to evaluate two competing inference engines (e.g., vLLM vs. SGLang) on real-world multi-turn conversational traces?
- **Short Answer**: Design a benchmarking framework that replays real-world traffic profiles:
- **Dataset**: Use a representative dataset of multi-turn conversational traces (e.g. ShareGPT).
- **Traffic Profile**: Replay requests using a Poisson process to simulate realistic arrival intervals.
- **Metrics**: Measure TTFT, TPOT, and overall TPS.
- **Controls**: Ensure both engines run on identical hardware, using identical weights, quantization parameters, and temperature configurations.
- **Key Interview Points**: Poisson request distribution, representative test traces, latency/throughput metrics, control variables.
- **Technical Intuition**: Replaying requests with realistic prompt-to-response length distributions is critical because prefill-to-decode ratios determine scheduler efficiency. Static mock sequences (e.g. 512 in, 512 out) fail to test prefix cache evictions or continuous batching scheduler performance under dynamic loads.
- **Production Perspective**: Run benchmarks under varying levels of concurrency to identify the saturation point where engines begin preemption or fail to meet SLAs.
- **Follow-up**: *Why is the Poisson process preferred for arrival times?* (Because it models independent, random request arrivals, which matches the behavior of real-world user traffic).